Aconcagua CFD

User Guide

Drag of the Aconcagua rocket with OpenFOAM v2412:
geometry, mesh, cases, sweeps, measurement and validation

Document	ACG-CFD-UG-001
Revision	A
Date	2026-10-18
Repository	`github.com/vnemegabriel/CFD-ITBA-ROCKETRY`
Applies to	branch `sweep/nemepad`, `flightModel/`
Organization	ITBA Rocketry

Document control

Revision history

Rev.	Date	Change
A	2026-10-18	First issue. Replaces the Markdown files `WORKFLOW.md`, `MESH.md`, `SOLVERS.md`, `VALIDATION.md` and `MEASUREMENTS.md` of `flightModel/docs/`. Adds the chapters on the comparison with Nektar++, the flight data and troubleshooting.

Status of the template

CAUTION

The case templates are not validated. Do not publish a result before you do the procedures in Chapter 11. The supersonic template has an open defect (Section 12.2).

Maintenance of this document

- This guide is the only user documentation of `flightModel/`. Do not write the same information in another file.
- Change the source in `flightModel/docs/guide/` and compile it with `make` (Section 2.4). Commit the source and the PDF together.
- When a change to a script changes a procedure, change the procedure in this guide in the same commit.
- Increase the revision letter and add a line to the revision history for each issue.

Contents

Document control i

About this guide xi

1 Overview 1
- 1.1 Purpose of the system . . . 1
- 1.2 Process . . . 1
- 1.3 Two solvers . . . 2
- 1.4 Directory structure . . . 2
- 1.5 Computers . . . 3

2 Installation and setup 5
- 2.1 Requirements . . . 5
- 2.2 Prepare the shell . . . 5
- 2.3 Prepare the computer for a sweep . . . 6
- 2.4 Build this guide . . . 6

3 Geometry 9
- 3.1 Requirements . . . 9
  - 3.1.1 Solid names that are not in the list . . . 9
- 3.2 makeRocket.py . . . 10
  - 3.2.1 Properties of the surface . . . 10
- 3.3 Use a new geometry . . . 11

4 Mesh 13
- 4.1 Mesh generator . . . 13
- 4.2 Make a mesh . . . 13
  - 4.2.1 Command . . . 13
  - 4.2.2 Procedure . . . 14
  - 4.2.3 Steps that Allmesh does . . . 14
  - 4.2.4 Result . . . 16
  - 4.2.5 Function of the .fms file . . . 16
- 4.3 meshDict . . . 16
  - 4.3.1 Global entries . . . 16
  - 4.3.2 localRefinement . . . 16
  - 4.3.3 objectRefinements . . . 17
  - 4.3.4 boundaryLayers . . . 18
  - 4.3.5 edgeMeshRefinement . . . 19
- 4.4 Domain . . . 20

4.4.1 Half mesh . . . 20
4.5 createPatchDict . . . 21
4.6 meshInfo . . . 21
4.7 Examine a mesh . . . 22

5 Cases **23**
5.1 Make a case . . . 23
5.1.1 Command . . . 23
5.1.2 Steps that newCase.sh does . . . 24
5.2 Run a case . . . 24
5.2.1 Command . . . 24
5.2.2 Steps that Allrun does . . . 25
5.2.3 Process binding . . . 26
5.2.4 End of a run . . . 26
5.2.5 Continue a run . . . 26
5.2.6 Start a run again from zero . . . 27
5.3 Start from another run . . . 27
5.4 Refine the mesh near the fins (optional) . . . 27

6 Sweeps **29**
6.1 Make the meshes . . . 29
6.2 Run the cases . . . 29
6.2.1 Start a sweep . . . 30
6.2.2 Stop a sweep . . . 30
6.3 The sweep of record . . . 31
6.3.1 Computers . . . 31
6.4 Flight conditions . . . 32
6.5 Do the sweep again on another computer . . . 32
6.6 Typical studies . . . 33
6.6.1 Mach sweep . . . 33
6.6.2 Mesh convergence . . . 33
6.6.3 Angle-of-attack sweep . . . 33
6.6.4 Finer fins . . . 33
6.7 Rules . . . 33

7 Solvers and models **35**
7.1 Summary . . . 35
7.1.1 Why two solvers . . . 35
7.2 Files of the template . . . 36
7.3 Flight condition . . . 37
7.3.1 flowConditions . . . 37
7.3.2 flowDerived . . . 37
7.4 Equations and numerical method of rhoCentralFoam . . . 38
7.5 Local time step, Courant ramp and stop criterion . . . 38
7.5.1 Local time step . . . 38
7.5.2 Courant ramp (central only) . . . 39
7.5.3 config.json . . . 39
7.5.4 Stop criterion . . . 40
7.6 Turbulence . . . 40
7.6.1 Wall treatment . . . 40

7.7 Thermophysical properties . . . 41
7.8 Temperature limits . . . 42
7.9 Boundary conditions . . . 42
7.10 Function objects . . . 43
7.11 Discretization schemes . . . 43
7.12 Linear solvers . . . 44
7.13 Origin of the configuration . . . 44
7.14 rhoPimpleFoam template . . . 45
7.14.1 Schemes . . . 45
7.14.2 PIMPLE . . . 46
7.14.3 Tests on h1 (2026-10-12) . . . 46
7.14.4 Two solvers at the same Mach number . . . 47
7.15 Start of the supersonic cases . . . 47

8 Results and measurement **49**
8.1 Outputs of a run . . . 49
8.2 run.info . . . 50
8.3 Measurement of the computer . . . 51
8.3.1 Requirements . . . 52
8.3.2 Procedure to get a comparable cost . . . 52
8.3.3 Do not divide by the clock frequency . . . 53
8.3.4 Runs without this measurement . . . 53
8.4 Monitor a run . . . 53
8.5 Table of a sweep . . . 53
8.6 Disk . . . 54
8.7 Notification . . . 54

9 Comparison with Nektar++ **55**
9.1 What must be the same . . . 55
9.2 What to compare . . . 56
9.3 How to measure in the same way . . . 56

10 Flight data **57**
10.1 Command . . . 57
10.2 Method . . . 57
10.3 Results . . . 59
10.4 Angle of attack in the flight . . . 59
10.5 Uncertainty . . . 60

11 Validation **61**
11.1 Checks on each run . . . 61
11.2 Subsonic flow: Barrowman . . . 62
11.2.1 Geometry data . . . 62
11.2.2 Normal force and centre of pressure . . . 62
11.2.3 Procedure . . . 62
11.2.4 Drag at zero angle of attack . . . 63
11.3 Transonic and supersonic flow . . . 63
11.4 Mesh convergence . . . 63
11.4.1 Procedure . . . 63
11.4.2 Equations . . . 64

11.5 Record 64

12 Troubleshooting **65**
12.1 Symptoms and remedies 65
12.2 Central template: drag increases without limit 66
12.3 mapFields on the same mesh 67
12.4 The Courant number in the log 67
12.5 Decimal comma in scripts 67
12.6 Rules for a change to the scripts 67

A Quick reference **69**
A.1 Commands 69
A.2 Run names 69
A.3 Key values 70

B Glossary **71**
B.1 Terms and abbreviations 71
B.2 Symbols 71

List of Figures

1.1 Process from the geometry to the results. 1

4.1 Steps of `Allmesh`. 15

5.1 How `newCase.sh` assembles a case from the templates and a mesh. 24
5.2 Steps of `Allrun`. 25

6.1 Chains of runs. Each arrow is an `--init`. 31

8.1 Measurement during the solver and its record in `run.info`. 51

10.1 Calculation of the flight drag coefficient. 58
10.2 C_D of the flight (coast) against the runs of the first sweep (ISA conditions, `~/runs.sin-medicion`) and OpenRocket. 60

List of Tables

1.1 Contents of `flightModel/`. 2

2.1 Software and hardware requirements. 5

3.1 Requirements for the STL file. 9
3.2 Effects of a solid name that is not in the list. 9
3.3 Parts of the Aconcagua geometry. 10
3.4 Properties of the surface and the measurements that set them. 11

4.1 Cell size of each level with `maxCellSize 1.6`. 13
4.2 Arguments of `Allmesh`. 13
4.3 Refinement regions (values for the Aconcagua). 17
4.4 Offset of the wake from the axis at the end of each wake region. 17
4.5 Entries of `boundaryLayers`. 18
4.6 Meshes of the sweep and their first cell. 19
4.7 Estimate of y^+ at the first cell centre, $y = 0.05\,mm$. 19
4.8 Limits of the domain. 20
4.9 Entries of `meshInfo`. 21
4.10 Measured quality of the meshes of the sweep. 22

5.1 Options of `newCase.sh`. 23
5.2 Flow angles that each mesh sector permits. 26

6.1 The sweep of record. 31
6.2 Flight conditions of the sweep. 32

7.1 Summary of the templates. 35
7.2 Files of `case-central/`. 36
7.3 Entries of `flowConditions` and the values of the template. 37
7.4 Values of `flowDerived` at two sweep conditions (`Ti` $= 0.005$, `nuRatio` $= 5$). . . 37
7.5 Courant ramp of `case-central/`. 39
7.6 Keys of `config.json`. 39
7.7 Constants of the gas. 41
7.8 Function objects. 43
7.10 Base pressure and temperature during the pimple start, $h1$, $M = 1.8$. 47

8.1 Outputs of a run. `<t0>` is the start time: 0 for a new run. 49
8.2 Samples for the comparison with another solver (function object `samples`). . . 50
8.3 Entries of `run.info`. 50
8.4 Block `measured` of `run.info`. 52

9.1 Items that the Nektar++ run must reproduce. 55

10.1 C_D of the flight and of the CFD at $\alpha = 0$ (first sweep, ISA conditions, 2026-10-17). 59

11.1 Checks on each run. 61

12.1 Symptoms, causes and remedies. 65
12.2 Pimple start against the central run, $h0$, $M = 1.8$. 67

About this guide

Purpose

This guide tells you how to calculate the drag coefficient of the Aconcagua rocket as a function of the Mach number with OpenFOAM. It gives the procedures, the reference data of each dictionary, the models, the measurements that each run records, and the procedures that validate the results.

Audience

The guide is for members of ITBA Rocketry who run or change the CFD cases. You must know how to use a Linux shell. You must know the basic concepts of compressible flow, RANS turbulence models and the finite-volume method. You do not have to know the OpenFOAM source code.

Scope

The guide covers the directory `flightModel/` of the repository: the geometry script, the mesh scripts, the two case templates, the sweep scripts, the post-processing scripts and the flight data. It does not cover the directory `fin_flutter/`.

How to use this guide

If you want to	Read
Make the first run	Chapters 1 and 2, then Sections 5.1 and 5.2
Run the sweep of record	Chapter 6
Change the geometry or the mesh	Chapters 3 and 4
Know why the solver is set up as it is	Chapter 7
Find a result or a measurement	Chapter 8
Compare a run with Nektar++	Chapter 9
Compare a run with the flight	Chapter 10
Validate a result	Chapter 11
Correct a problem	Chapter 12
Find a command quickly	Appendix A

Conventions

Convention	Meaning
`mesh/Allmesh`	A file, a directory, a command, a dictionary entry or a value.
`<mesh-dir>`	A value that you must supply.
`[stl-file]`	An optional argument.
`~/meshes`, `~/runs`	The default directories of the meshes and of the runs.
Paths without a leading `/`	Relative to `flightModel/`, except where the text says otherwise.
$h0$, $h1$, $h2$, $h3$	The four meshes of the sweep (Table 4.6).
`m08-a04-h1`	A run: Mach 0.8, angle of attack $4°$, mesh $h1$.

Procedures are numbered steps. Each step gives one action. A result that you must see after a step follows the step.

Notices

CAUTION

A CAUTION tells you about a condition that can damage data, give an incorrect result without a message, or waste days of computer time.

NOTE

A NOTE gives information that helps you to do a step or to understand a result.

Writing standard

The structure of this guide follows ISO/IEC/IEEE 26514 [1]: concepts, procedures and reference information are in different sections. The text follows the writing rules of ASD-STE100 Simplified Technical English [2]: short sentences, one instruction in each sentence, the active voice and one meaning for each word. Technical names (*mesh*, *solver*, y^+, *wall function*) stay as the OpenFOAM documentation writes them.

Related documents

Document	Contents
`CLAUDE.md`	Rules for the people and the agents who change the repository.
`flightModel/case-central/README.md`, `case-pimple/README.md`	Short description of each template, with references to this guide.
`flightModel/flightData/cdFlight.py`	Source of the flight analysis of Chapter 10.
OpenFOAM v2412 user guide	`https://www.openfoam.com/documentation/user-guide`

Chapter 1

Overview

1.1 Purpose of the system

The system calculates the drag coefficient C_D of the Aconcagua rocket as a function of the Mach number with OpenFOAM v2412. The output of record is the curve $C_D(M)$ at $M = 0.3$, 0.6, 0.8, 1.2 and 1.8, at angles of attack $\alpha = 0°$, $4°$ and $8°$, on three meshes for a mesh-convergence study. The flight conditions are those of the coast phase of the IREC flight in Midland (Section 6.4).

The mesh must resolve these flow features, in this order of priority:

1. The shock waves, for $M \geq 0.8$.
2. The base wake. The base drag is a large part of the drag of a rocket.
3. The wall friction. It depends on y^+.

The tip vortex of the fins is not a priority. Do not add cells for it.

1.2 Process

Figure 1.1 shows the process. Each step is a script, and each script has a section in this guide.

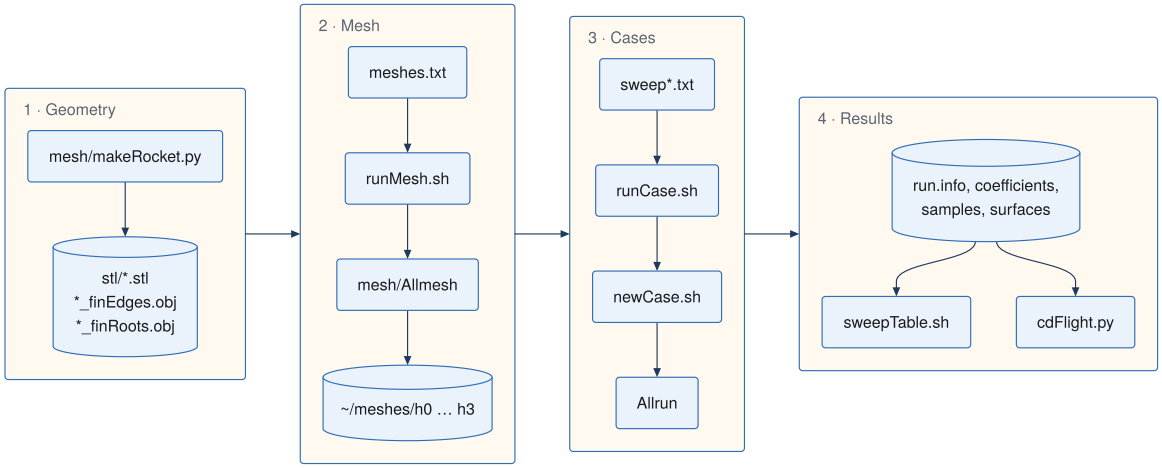


Figure 1.1: Process from the geometry to the results.

1. `mesh/makeRocket.py` writes the geometry as an STL file and two edge files (Chapter 3).

2. `mesh/Allmesh` makes one mesh. `runMesh.sh` makes all the meshes of `meshes.txt` (Chapter 4 and Section 6.1).

3. `newCase.sh` makes one case from a template, a mesh and a flight condition. `common/Allrun` runs it (Chapter 5). `runCase.sh` does the two steps for each line of a sweep file (Chapter 6).

4. Each run writes its coefficients, its samples and a record of the machine (Chapter 8). `post/sweepTable.sh` collects a sweep in one table, and `flightData/cdFlight.py` compares it with the flight (Chapter 10).

1.3 Two solvers

The case templates use two solvers, with the change at $M = 1$:

Template	Solver	Use
`case-pimple/`	`rhoPimpleFoam`	$M < 1$, and the first run of each $M > 1$ chain
`case-central/`	`rhoCentralFoam`	$M > 1$

Chapter 7 gives the reasons and the models.

1.4 Directory structure

Table 1.1: Contents of `flightModel/`.

Path	Contents
`mesh/makeRocket.py`	Writes the geometry: the STL file and its edge files (Section 3.2).
`mesh/stl/`	The geometry files. `Aconcagua_biconvex117_roll45.stl` is the default (fins at $45°$).
`mesh/Allmesh`	Makes one mesh (Section 4.2).
`mesh/snapPlane.py`	Sets $z = 0$ on the cut of a half mesh.
`mesh/system/`	`meshDict`, `createPatchDict`, `createPatchDictHalf`, `topoSetDictHalf`, and the dictionaries that the mesh applications need.
`mesh/meshInfo`	Reference values and fin geometry of the mesh (Section 4.6).
`case-central/`	Template for `rhoCentralFoam`. It also holds the files that the two templates share.
`case-pimple/`	Template for `rhoPimpleFoam`. It holds only the files that differ.
`common/Allrun`, `common/Allrefine`, `common/machineLog.py`	Scripts that `newCase.sh` copies into each case.
`newCase.sh`	Makes one case (Section 5.1).

Path	Contents
`runMesh.sh`, `meshes.txt`	Make the meshes of the sweep (Section 6.1).
`runCase.sh`, `sweep*.txt`	Make and run the cases of a sweep (Section 6.2).
`post/monitor.sh`	Plots a run while it runs (Section 8.4).
`post/sweepTable.sh`	Writes a sweep as a CSV table (Section 8.5).
`post/firstLayer.py`	Measures the first cell on the walls of a mesh (Section 4.3.4).
`flightData/`	The TeleMega data of the flight and `cdFlight.py` (Chapter 10).
`docs/guide/`	This guide.

1.5 Computers

	`newton`	`nemepad`
CPU	2 × Xeon E5-2660 v4, 2 sockets of 14 cores	Ryzen 7 250, 1 socket of 8 cores, 16 threads
Memory	ECC	28 GB, no ECC
Open MPI	4.1	5.0.10
Locale	`es_ES`: `awk` writes decimal commas (Section 12.5)	`en`
Sweep files	`sweepSubsonic.txt`, `sweepSupersonic.txt`, one on each socket	`*.nemepad.txt`, one file at a time
Status (2026-10-18)	Out of service: the memory module DIMM_A1 stops the computer under load	In use

Chapter 2

Installation and setup

2.1 Requirements

Table 2.1: Software and hardware requirements.

Item	Requirement
OpenFOAM	ESI v2412, with cfMesh (`cartesianMesh`).
Open MPI	The version of the OpenFOAM installation. 4.1 and 5.0 are tested.
`jinja2`	`jinja2-cli`. Install it with `pipx install jinja2-cli`.
Shell tools	`bash`, `rsync`, `awk`, GNU `sed`, `xz`, `/usr/bin/time`.
Python 3	Standard library only. `cdFlight.py` also needs `matplotlib`.
`perf`	Optional. Counts the CPU cycles of each run (Section 8.3).
`gnuplot`	Optional. For `post/monitor.sh`.
Memory	About 3 GB for each million cells. $h3$ (10.0 M cells) needs about 30 GB.
Disk	About 1 GB for each mesh of 2 M cells, and 0.4 GB for each run on it.

2.2 Prepare the shell

Do this step in each new shell, before you use a script:

```
source /usr/lib/openfoam/openfoam2412/etc/bashrc
```

If you do not do this step, all scripts stop with the message `source the OpenFOAM bashrc first`.

CAUTION

Put the mesh directories and the run directories outside `/mnt/c`. On WSL, that file system makes the input and output of OpenFOAM several times slower.

2.3 Prepare the computer for a sweep

A sweep runs for days. The cost of each run is a measurement (Section 8.3). Do these steps before you start a sweep. Steps 1 to 4 need `sudo`.

1. Set the CPU profile to `performance`. Other profiles decrease the clock frequency, and the cost of the run does not compare with other runs.

   ```
   echo performance | sudo tee /sys/firmware/acpi/platform_profile
   ```

   The setting is lost when the computer starts again.

2. Let `perf` count the cycles and the energy, permanently:

   ```
   echo kernel.perf_event_paranoid=0 | sudo tee /etc/sysctl.d/90-perf.conf
   sudo sysctl --system
   ```

   With `1`, `perf` counts the cycles but not the energy. With `2` or more, it counts neither.

3. On a laptop, make sure that the computer does not suspend when you close the lid. Set `HandleLidSwitch=ignore` in `/etc/systemd/logind.conf`, or keep this command open in a terminal for the duration of the sweep:

   ```
   systemd-inhibit --what=handle-lid-switch:sleep --why="CFD sweep" sleep infinity
   ```

4. Make sure that the memory has sufficient cooling. Keep the air inlets open and lift the rear of a laptop. On `nemepad` a memory module reached 83 °C (limit 85 °C) with the fans in automatic mode.

5. Stop other heavy programs. Two runs on one socket share the memory bandwidth.

CAUTION

On a ThinkPad, a reload of the module `thinkpad_acpi` (for example to control the fans) can set the CPU profile to `custom`, at about 2 GHz. Do step 1 again after a reload.

2.4 Build this guide

The guide needs LuaLaTeX, BibTeX, the TeX Gyre fonts and the packages of TeX Live `latex-extra`. The diagrams need Node.js 20 or later, the npm package `pele` and Google Chrome.

1. Go to the directory of the guide:

   ```
   cd flightModel/docs/guide
   ```

2. If you changed a diagram, install Pele once and render the diagrams:

   ```
   npm install
   make figures
   ```

   `figures/render.mjs` renders each `figures/src/*.mmd` (Mermaid) to SVG with Pele, and prints it to `figures/pdf/*.pdf` with headless Chrome. Set `CHROME` to use another Chrome executable.

3. Compile the guide:

```
make
```

   Result: `aconcagua-user-guide.pdf`. `make` runs `lualatex`, `bibtex` and `lualatex` two more times.

4. Commit the source, `figures/pdf/` and the PDF together.

Chapter 3

Geometry

3.1 Requirements

Table 3.1: Requirements for the STL file.

Item	Requirement	Reason
Format	ASCII STL	A binary STL has no solid names. All the rocket then becomes one patch.
Solid names	`nosecone`, `body`, `boattail`, `fins`	`Allmesh` changes the first three to `cone`, `walls`, `tail`. The dictionaries and the cases use these four names.
Units	Metres	`meshDict` and `meshInfo` use metres.
Position	Nose tip at the origin, axis along $+x$	The refinement regions, the fin region in `meshInfo` and the moment reference `CofR (0 0 0)` use this position.
Edge files	`<name>_finEdges.obj` and `<name>_finRoots.obj` next to the STL	`Allmesh` copies them for `edgeMeshRefinement` (Section 4.3.5).

3.1.1 Solid names that are not in the list

`Allmesh` does not stop if a solid has a different name. The patch keeps the name of the solid. Table 3.2 gives the effects.

Table 3.2: Effects of a solid name that is not in the list.

Step	Effect
`cartesianMesh`, `localRefinement`	An entry without a patch gives only a warning. The patch gets no surface refinement. The cell size near the wall is then that of the volume region (50 mm in `nearField`), not 6.25 mm.
`cartesianMesh`, `boundaryLayers`	The patch gets only the global layers (Section 4.3.4).

Step	Effect
Case, `0.orig/`	The boundary conditions use the names `cone`, `walls`, `tail`, `fins`. A patch with a different name has no boundary condition, and the solver stops.
Case, `forceCoeffs`	`wallPatches` in `meshInfo` does not contain the patch. Its force is not in the coefficients.

Before you make a mesh from a new STL file, examine the solid names:

```
grep -E '^ *solid ' mesh/stl/<file>.stl
```

3.2 makeRocket.py

`mesh/makeRocket.py` writes the geometry from its dimensions. It writes three files: the STL file, `<name>_finEdges.obj` (the leading and trailing edges of the fins) and `<name>_finRoots.obj` (the root of each fin). `python3 mesh/makeRocket.py -h` gives the options. A file with one `name value` on each line can give the dimensions (`--dims`).

To write the geometry of the full mesh (fins at $0°$):

```
python3 mesh/makeRocket.py mesh/stl/Aconcagua_biconvex117.stl
```

To write the geometry of the half mesh (fins at $45°$, the default of `Allmesh`):

```
python3 mesh/makeRocket.py mesh/stl/Aconcagua_biconvex117_roll45.stl --finRoll 45
```

`--finRoll 45` turns the fins to $45°$, $135°$, $225°$ and $315°$. The plane $z = 0$ then passes between two fins. The STL file is always the whole rocket: the half mesh is cut from the full mesh (Section 4.4.1).

Table 3.3: Parts of the Aconcagua geometry.

Part	Shape	Solid name
Nose	Von Kármán, 0.8 m long	`nosecone`
Body	Cylinder, $d = 0.151\,m$	`body`
Fins	4 fins, biconvex section of constant thickness 11.7 mm, edges rounded with 1 mm, a fillet of 3 mm all round the root	`fins`
Boattail	Cone to $d = 0.110\,m$, and the base. Shoulder and base edge rounded with 3 mm	`boattail`

3.2.1 Properties of the surface

The script writes a file only if the surface is closed and all the normals point out. The properties in Table 3.4 come from measurements with `cartesianMesh`. Do not change them.

Table 3.4: Properties of the surface and the measurements that set them.

Property	Reason, measured with the same `meshDict`
Nose and body + fins + boattail are two closed shells that touch on coplanar caps. The rings of the two shells are turned by half a facet.	With a sharp shoulder, one continuous hull gave 12 to 87 negative cell volumes at the shoulder. With the shoulder rounded (`--shoulderRadius`, 3 mm) the continuous hull gave 0 on $h1$. The boattail must be in the body shell, because the root fillet of the trailing edge runs 3 mm onto the cone.
Body and fins are one shell: the fin root is cut into the cylinder.	With the fins as separate shells sunk into the body, the fin-body junction was not an edge of the surface. The layers of the two surfaces met at one node in the corner.
The leading edge, the trailing edge and the tip of the fins have a radius (`--finEdgeRadius`, 1 mm).	With sharp edges, the layers of the two fin faces met at the edge: maximum skewness 25 to 33, non-orthogonality up to $176°$. With the radius: skewness 3.7, non-orthogonality $77°$ to $82°$.
A fillet of 3 mm all round each fin root (`--finRootRadius`).	Without it, the fin face and the body met at about $90°$, and the fin layers and the body layers closed to one node along the root. With it, on $h1$: non-orthogonality $81.9° \rightarrow 67.7°$, skewness $3.58 \rightarrow 1.89$, smallest cell $2.6 \rightarrow 6.9 \times 10^{-13}$ m^3.
The base edge is rounded with 3 mm (`--baseEdgeRadius`).	
The root trailing edge is on the shoulder (`--finFromBase` = `--tailLength`).	
44 facets around the axis (`--nTheta`).	With 96 facets the continuous hull had more negative volumes.

The edge radius is in the section parallel to the axis. Normal to the swept leading edge ($57°$), it is approximately 0.5 mm. The section is the convex hull of the biconvex arcs and a circle of the radius at each edge. The profile is thus a little thicker in approximately the first 80 mm of the chord.

3.3 Use a new geometry

1. Write the STL file with `makeRocket.py` (Section 3.2), or put it in `mesh/stl/` with its two edge files.

2. Make sure that the STL file agrees with Table 3.1.

3. Change the six distances of `surfaceGenerateBoundingBox` in `Allmesh` (Section 4.4).

4. Write `lBody` and `rBody` of the new rocket in `mesh/meshInfo`. The refinement regions scale with them (Section 4.3.3).

5. Change the other entries of `mesh/meshInfo` (Section 4.6).
6. Make the mesh: `mesh/Allmesh <mesh-dir> mesh/stl/<file>.stl`.
7. Remove the old meshes. `runMesh.sh` does not compare the geometry (Section 6.1).

Chapter 4

Mesh

4.1 Mesh generator

The mesh generator is cfMesh (`cartesianMesh`). It makes an octree mesh from one maximum cell size. All other sizes are refinement levels. Each level divides the cell size by 2:

$$\Delta = \frac{\texttt{maxCellSize}}{2^{\,\text{level}}}$$

Table 4.1: Cell size of each level with `maxCellSize 1.6`.

Level	2	4	5	6	7	8	9
Size [mm]	400	100	50	25	12.5	6.25	3.13
Level	10	11					
Size [mm]	1.56	0.78					

A level has an effect only on its patch or its region. If a level is incorrect, the cells in that region are too large or too small. No other region changes.

4.2 Make a mesh

4.2.1 Command

```
mesh/Allmesh <mesh-dir> [stl-file] [--maxCellSize <m>] [--nLayers <n>]
             [--thicknessRatio <r>] [--full]
```

Table 4.2: Arguments of `Allmesh`.

Argument	Default	Function
`<mesh-dir>`	mandatory	Directory of the new mesh. It must not exist.
`[stl-file]`	`mesh/stl/Aconcagua_biconvex117_roll45.stl`; with `--full`, `Aconcagua_biconvex117.stl`	Geometry (Section 3.1).

Argument	Default	Function
`--maxCellSize <m>`	`meshDict` (1.6)	Sets `maxCellSize` in the copy of `meshDict`.
`--nLayers <n>`	`meshDict` (10)	Sets `nLayers` of `cone`, `walls` and `tail`.
`--thicknessRatio <r>`	`meshDict` (1.2)	Sets `thicknessRatio` of `cone`, `walls` and `tail`.
`--full`	half mesh	Mesh of the whole rocket, fins at $0°$, $90°$, $180°$, $270°$. Without it, the mesh is the half $z \geq 0$ (Section 4.4.1).

4.2.2 Procedure

1. Run `mesh/Allmesh <mesh-dir>` with the options of the mesh.

2. Read the summary at the end of the output. It gives the number of cells, the cell types, the maximum non-orthogonality, the maximum skewness and the failed checks.

3. If the output shows `negative cell volumes`, do not use the mesh. Change `meshDict` and make a new mesh in a new directory.

4. Measure the first cell on the walls (Section 4.3.4):

```
./post/firstLayer.py <mesh-dir>
```

5. Do the checks of Section 4.7.

CAUTION

A mesh with negative cell volumes is not usable. The other checks of `-allGeometry` that fail on these meshes (small determinant in the layer cells, face tets, concave cells) are of a different category and do not stop the use of the mesh.

4.2.3 Steps that Allmesh does

`Allmesh` does the steps of Figure 4.1 in the mesh directory. If an application writes `FOAM FATAL` in its log, `Allmesh` shows the error and stops. `Allmesh` reads each log, because `runApplication` returns 0 also when the application fails.

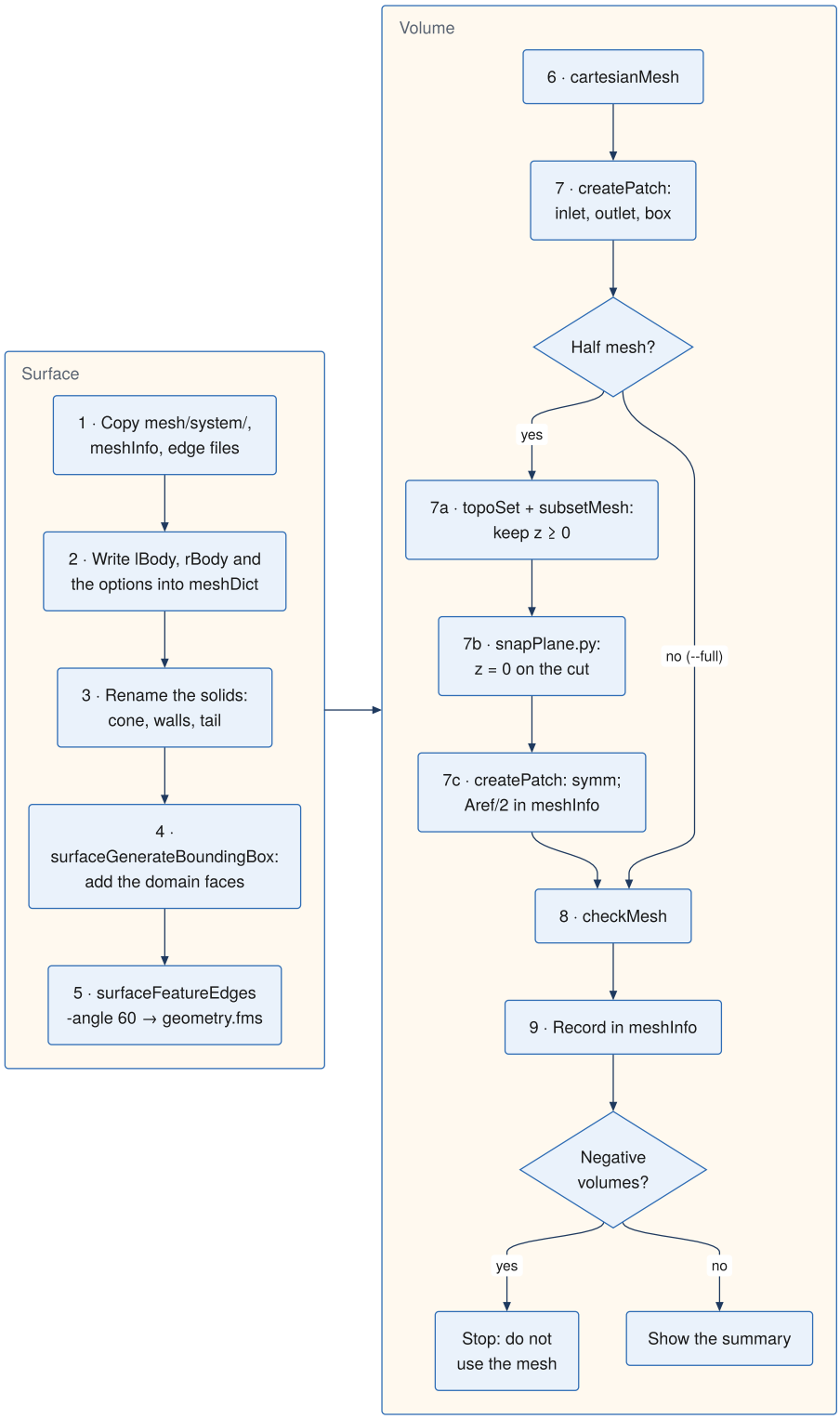


Figure 4.1: Steps of `Allmesh`.

4.2.4 Result

Item	Contents
`<mesh-dir>/constant/polyMesh/`	The mesh.
`<mesh-dir>/system/`	A copy of `mesh/system/` with the options applied. Each mesh keeps the `meshDict` that made it.
`<mesh-dir>/meshInfo`	A copy of `mesh/meshInfo`, with the record of the mesh (Section 4.6).
`<mesh-dir>/meshHash`	Fingerprint of the mesh. `newCase.sh` writes it the first time that a case uses the mesh (Section 9.1).
`<mesh-dir>/log.*`	One log for each application.

`Allmesh` does not write into a directory that exists. To make the mesh again, remove the directory or use a new name.

4.2.5 Function of the .fms file

An STL file contains only triangles and solid names. It does not contain edges. The `.fms` file contains the triangles, the patches and the feature edges. `surfaceFeatureEdges` marks an edge as a feature edge when the angle between the two adjacent triangles is more than $60°$. `cartesianMesh` then moves the mesh vertices onto these edges.

The rocket has no sharp edge: the fin edges, the fin root, the shoulder and the base edge are rounded, and each round has facets of less than $30°$. With a limit of $30°$, small triangles where the root fillet meets the body gave 172 false feature edges, and `cartesianMesh` bent the cells along them. With $60°$, only the edges of the domain box (12) and of the nose (88) remain.

4.3 meshDict

File: `mesh/system/meshDict`. Each mesh directory has its own copy.

4.3.1 Global entries

Entry	Value	Function
`surfaceFile`	`"geometry.fms"`	Surface that `Allmesh` makes in step 5.
`maxCellSize`	`1.6`	Maximum cell size in m. All levels refer to it.

4.3.2 localRefinement

Surface refinement for each patch. `refinementThickness` is the distance from the wall in which the cells have the level of the patch. Outside this distance, the cell size increases to `maxCellSize` one level at a time.

Patch	Level	Size	Thickness	Reason
`cone`	8	6.25 mm	0.060 m	The bow shock is in this band.
`walls`	8	6.25 mm	0.050 m	Body boundary layer.
`tail`	8	6.25 mm	0.050 m	The expansion on the boattail sets the base pressure.
`fins`	9	3.13 mm	0.020 m	Fin chord.

4.3.3 objectRefinements

Volume refinement in regions. A `box` has a centre and three lengths. A `cone` has two points on the axis and a radius at each point.

Table 4.3: Refinement regions (values for the Aconcagua).

Region	Type	Level	Size	Extent
`shoulder`	hollow cone	9	3.13 mm	x from 2.81 to 2.85 m, r from 0.065 to 0.090 m: the body-boattail shoulder.
`baseEdge`	hollow cone	10	1.56 mm	x from 2.94 to 2.97 m, r from 0.045 to 0.065 m: the base edge, where the base shear layer starts.
`nearField`	box	5	50 mm	x from -0.20 to 3.20 m, y and z $\pm 0.50\,m$.
`baseWake`	cone	8	6.25 mm	x from 2.94 to 3.30 m, radius 0.08 m.
`nearWake`	cone	7	12.5 mm	x from 2.90 to 3.80 m, radius 0.14 m.
`midWake`	cone	6	25 mm	x from 2.85 to 5.00 m, radius 0.26 m.
`wake`	box	4	100 mm	x from 1.0 to 9.0 m, y and z $\pm 0.80\,m$.
`farWake`	box	2	400 mm	x from 1.0 to 27.0 m, y and z $\pm 2.0\,m$.
`noseTip`	sphere	9	3.13 mm	Centre $(0, 0, 0)$, radius 0.040 m.
`noseApex`	sphere	11	0.78 mm	Centre $(0, 0, 0)$, radius 0.010 m.

`noseTip` has the level of the fins. `noseApex` makes the curve of the tip and keeps the layers thinner than the radius of the nose. Without `noseApex`, the full mesh $h2$ had 13 negative cell volumes at the tip.

The wake regions decrease the cell size one level at a time behind the base. They are cones on the x axis. At an angle of attack, the wake moves $\tan\alpha$ for each metre towards $+y$ (Table 4.4).

Table 4.4: Offset of the wake from the axis at the end of each wake region.

Region	Radius	Length behind the base	Offset at $4°$	Offset at $8°$
`baseWake`	0.08 m	0.36 m	0.03 m	0.05 m
`nearWake`	0.14 m	0.85 m	0.06 m	0.12 m
`midWake`	0.26 m	2.1 m	0.15 m	0.30 m

At $4°$ the wake stays inside the three regions. At $8°$ it leaves `midWake` approximately 1.85 m behind the base.

Each region is written as `#eval{ <value>*$sx }` along x and `#eval{ <value>*$sr }` across, with s_x = `lBody`$/2.955$ and s_r = `rBody`$/0.0755$. `Allmesh` writes `lBody` and `rBody` from `meshInfo` into the copy of `meshDict`. A rocket of another size thus gets the regions moved and scaled with it.

CAUTION

`foamDictionary -set` evaluates the `#eval` entries and writes the numbers. `Allmesh` writes `lBody` and `rBody` with `sed` before the first `foamDictionary -set`. Do not set `lBody` with `foamDictionary` in a copy that was already written.

4.3.4 boundaryLayers

cfMesh extrudes the layers from the surface mesh. The first cell thickness t_1 thus comes from the surface cell size h, `nLayers` n and `thicknessRatio` r. The dictionary does not set a y^+ target.

Table 4.5: Entries of `boundaryLayers`.

Entry	Value	Function
`nLayers`	`1`	Layers on the patches without an entry in `patchBoundaryLayers`: the six faces of the domain.
`patchBoundaryLayers/cone, walls, tail`	`nLayers 10, thicknessRatio 1.2`	Layers on the body. `Allmesh` options change them.
`patchBoundaryLayers/fins`	`nLayers 3, thicknessRatio 1.2, allowDiscontinuity 1`	More layers tangled the cells at the leading edge of the old 6 mm fin. This limit was not measured again on the 11.7 mm fin with rounded edges.
`optimiseLayer, untangleLayers`	`1, 1`	Smooth and repair the layer cells.
`optimisationParameters`	`nSmoothNormals 5, maxNumIterations 5, featureSizeFactor 0.3, reCalculateNormals 1, relThicknessTol 0.03`	Parameters of the layer optimization.

CAUTION

cfMesh splits the layer only on the patches that have an entry in `patchBoundaryLayers`. With only the global `nLayers 10`, the body got no layers, and the faces of the domain got 10. Keep an entry for each wall patch.

The layer rule is

$$t_1 = h\,\frac{r-1}{r^n-1},$$

if the whole layer were one surface cell thick. `cartesianMesh` makes it thinner: the measured first cell is 0.41 times the rule. The rule thus keeps t_1 equal on all meshes, but it does not give

its value. `post/firstLayer.py <mesh-dir>` measures it. It gives y, the distance from the cell centre to the wall face (as `yPlus` uses it), and $t_1 = 2y$ for each wall patch, weighted by the face area.

Table 4.6: Meshes of the sweep and their first cell.

Mesh	`maxCellSize`	h, body	n	r	t_1, rule	t_1, measured
$h0$	2.2627	8.84 mm	12	1.1878	0.241 mm	0.097 mm
$h1$	1.6	6.25 mm	10	1.2	0.241 mm	0.100 mm
$h2$	1.1314	4.42 mm	8	1.229	0.241 mm	0.096 mm
$h3$	0.8	3.13 mm	7	1.2	0.241 mm	0.101 mm

Thus $y \approx 0.05\,mm$ on the body. Where a region refines the wall, the layer is thinner: under `finRoots` (level 11) t_1 is approximately 0.014 mm, and on the boattail approximately 0.04 mm.

Table 4.7 gives y^+ from the flat-plate friction at the aft body ($x \approx 2.8\,m$), where y^+ is lowest.

Table 4.7: Estimate of y^+ at the first cell centre, $y = 0.05\,mm$.

M	u_τ/ν [1/mm]	y^+
0.3	∼110	∼5.5
0.6	∼230	∼11.5
0.8	∼330	∼16.5
1.2	∼550	∼27.5
1.8	∼950	∼47.5

From $M = 0.6$ to 1.2 the first cell is in the buffer layer, and at $M = 0.3$ at its lower limit. `nut` is `nutUSpaldingWallFunction`, which follows Spalding's law through the buffer layer (Section 7.6.1). These values are estimates. The measured mean y^+ on `walls` was 2.4 at $M = 0.3$, 6.9 at $M = 0.8$ and 16.6 at $M = 1.8$ on $h0$. Measure y^+ on each run (Section 8.1).

4.3.5 edgeMeshRefinement

Entry	Value	Function
`finEdges/edgeFile`	`"finEdges.obj"`	Leading and trailing edges of the fins.
`finEdges/additionalRefinementLevels`	`10`	1.56 mm with `maxCellSize 1.6`.
`finEdges/refinementThickness`	0.010 m	Width of the band around each edge.
`finRoots/edgeFile`	`"finRoots.obj"`	Root of each fin, from the leading to the trailing edge.
`finRoots/additionalRefinementLevels`	`11`	0.78 mm with `maxCellSize 1.6`.
`finRoots/refinementThickness`	0.008 m	Width of the band around each root.

CAUTION

`finRoots` also refines the body wall. The layer thickness follows the surface cell. At level 11 the first body layer near the root is approximately 0.014 mm, against 0.10 mm on the rest of the body, and y^+ there is approximately 7 times lower.

4.4 Domain

`Allmesh` step 4 sets the domain with `surfaceGenerateBoundingBox`, which takes six distances in metres from the bounding box of the rocket. The upstream and downstream distances are fixed in `Allmesh`. The lateral distances come from the STL file: `Allmesh` calculates them so that the side walls are at $|y| = |z| = 11.775\,m$ for any rocket. The box is thus symmetric about $z = 0$, which the half mesh needs (Section 4.4.1).

Table 4.8: Limits of the domain.

Direction	Distance from the rocket	Limit
Upstream $(-x)$	17.73 m	$x = -17.73\,m$
Downstream $(+x)$	38.415 m	$x = 41.37\,m$
Lateral $(\pm y, \pm z)$	from the STL file	$\pm 11.775\,m$

CAUTION

`Allmesh` calculates the lateral distances with `awk`, and fixes `LC_ALL=C`. With the locale `es_ES`, `awk` wrote `11,775` and the domain was incorrect (Section 12.5).

At $M = 1.8$ the Mach angle is $33.7°$. The Mach cone from the nose crosses the lateral boundary at approximately $x = 17.6\,m$. The pressure boundary condition on `box` thus must let the wave go out (Section 7.9).

4.4.1 Half mesh

At $\alpha = 0$ the flow has the symmetry of the fins. At an angle of attack in the x–y plane, the plane $z = 0$ stays a plane of symmetry. By default `Allmesh` meshes the whole rocket with the fins at $45°$ and keeps the half $z \geq 0$.

1. The box is symmetric about $z = 0$. The octree thus splits on $z = 0$, and the cells of the full mesh do not cross it, except approximately 0.1 % near the wall.

2. `topoSet` selects the cells with the centre at $z \geq 0$, and `subsetMesh` keeps them. The faces of the cut go to `oldInternalFaces`.

3. The smoothing of `cartesianMesh` leaves the cut up to 1.7 mm off the plane. `snapPlane.py` sets $z = 0$ on each point of the cut.

4. `createPatchDictHalf` makes the cut the patch `symm`, of type `symmetry`.

5. `meshInfo` gets `sector half`, `symmetryFraction 0.5` and `Aref` $= \pi r_{\text{body}}^2/2$. The coefficients are thus those of the full rocket. `Allrun` then permits α but not β.

The half mesh takes the meshing time and the memory of the full mesh, because `cartesianMesh` makes the full mesh.

CAUTION

Do not mesh $z \geq 0$ directly with `zMin` on $z = 0$. `cartesianMesh` then treats `zMin` as a domain face that touches the wall. It gives `zMin` the layers of the wall, and the wall layers fan out at the corner: 9 002 faces with interpolation weight below 0.05 on $h1$.

4.5 createPatchDict

`surfaceGenerateBoundingBox` makes six patches with fixed names. `createPatch` merges them:

New patch	Type	Source patches
`inlet`	`patch`	`xMin`
`outlet`	`patch`	`xMax`
`box`	`patch`	`yMin`, `yMax`, `zMin`, `zMax`
`symm` (half mesh)	`symmetry`	`oldInternalFaces`

The `.fms` file gives no type to its patches, so `cartesianMesh` makes all of them `wall`. `createPatch` makes the domain patches of type `patch`. Only `cone`, `walls`, `tail` and `fins` stay `wall`. The wall functions and the `yPlus` function object need the type `wall`. `Allmesh` reads the type of each patch in `wallPatches` and stops if a type is not `wall`.

4.6 meshInfo

File: `mesh/meshInfo`. `Allmesh` copies it into the mesh directory. `newCase.sh` copies it to `constant/meshInfo` in the case, where it replaces the `meshInfo` of the template.

Table 4.9: Entries of `meshInfo`.

Entry	Value	Written by	Function
`sector`	`full`	user, `Allmesh`	`quarter`, `half` or `full`. `Allrun` uses it to check the flow angles.
`symmetryFraction`	`1`	user, `Allmesh`	Fraction of the rocket in the mesh.
`fins`	`true`	user	`Allrefine` stops if it is not `true`.
`nCells`	—	`Allmesh`, `Allrefine`	Number of cells.
`Aref`	0.0179081 m^2	user, `Allmesh`	Reference area: πr_{body}^2, half of it for a half mesh.
`lRef`	0.151 m	user	Reference length: the body diameter.
`rBody`	0.0755 m	user	Body radius. Scales the regions across the axis.

Entry	Value	Written by	Function
`lBody`	2.955 m	user	Rocket length. Scales the regions along the axis.
`finRootR`, `finTipR`	0.075, 0.2355 m	user	Radii of the fin root and the fin tip. `finTipR` sets the regions of `Allrefine`.
`finTipSmear`	0.004 m	user	Width of the tip chamfer.
`finX0`, `finX1`	2.52953, 2.93031 m	user	x of the front and the rear of the fins.
`wallPatches`	`(cone walls tail fins)`	user	Patches in the force integration and in `wallShearStress`.
`maxCellSize`, `finLevel`, `nLayers`, `thicknessRatio`, `meshDir`	—	`Allmesh`	Record of the mesh.

NOTE

`Aref` must agree with `sector`. The `case-central/constant/meshInfo` file of the template has the values of a `quarter` mesh. `newCase.sh` replaces it, so these values have no effect.

4.7 Examine a mesh

1. Read the summary of `Allmesh`. There must be no negative cell volume.
2. Examine the patch names and types in `constant/polyMesh/boundary`. `cone`, `walls`, `tail` and `fins` must be `wall`.
3. Measure t_1 with `post/firstLayer.py`. The median on `walls` must agree with Table 4.6.
4. Open the mesh in ParaView. Look at the cells on the fin edges, on the base edge and on the boattail shoulder.
5. Compare the number of cells with Table 4.10.

Table 4.10: Measured quality of the meshes of the sweep.

Mesh	Cells	Neg. volumes	Skewness	Non-orth.	t_1, `walls`
$h0$	1 074 798	0	2.37	65.7°	0.097 mm
$h1$	2 198 939	0	1.72	66.0°	0.100 mm
$h2$	4 285 770	0	1.58	65.6°	0.096 mm
$h3$	10 020 768	0	1.41	65.9°	0.101 mm

Chapter 5

Cases

A case is one run: one template, one mesh and one flight condition. Make it with `newCase.sh` and run it with `Allrun`. `runCase.sh` does the two steps for each line of a sweep file (Chapter 6).

5.1 Make a case

5.1.1 Command

```
./newCase.sh <run-dir> --mesh <mesh-dir> [options]
```

Table 5.1: Options of `newCase.sh`.

Option	Function
`--mesh <mesh-dir>`	Mandatory. The path of a mesh that `Allmesh` made.
`--Minf <M>`	Free-stream Mach number.
`--pInf <Pa>`, `--Tinf <K>`	Free-stream static pressure and temperature.
`--alpha <deg>`, `--beta <deg>`	Angle of attack and angle of sideslip.
`--Ti <->`, `--nuRatio <->`	Free-stream turbulence intensity and ratio ν_t/ν.
`--np <N>`	Number of subdomains in `system/decomposeParDict`.
`--socket <S>`	Binds the run to socket `S`, one process per physical core (Section 5.2.3). `none` disables the binding.
`--refine tip`, `--refine fins`	Runs `Allrefine` on the new case (Section 5.4).
`--init <run-dir>`	Starts from the latest time of another run (Section 5.3).
`--solver central`, `--solver pimple`	Template `case-central/` (default) or `case-pimple/`.

The flow options change entries in `system/flowConditions`. The options that you do not give keep the values of the template (Table 7.3).

NOTE

For `newCase.sh`, `--mesh` and `--init` are paths. Only `runCase.sh` accepts a name without a path.

5.1.2 Steps that newCase.sh does

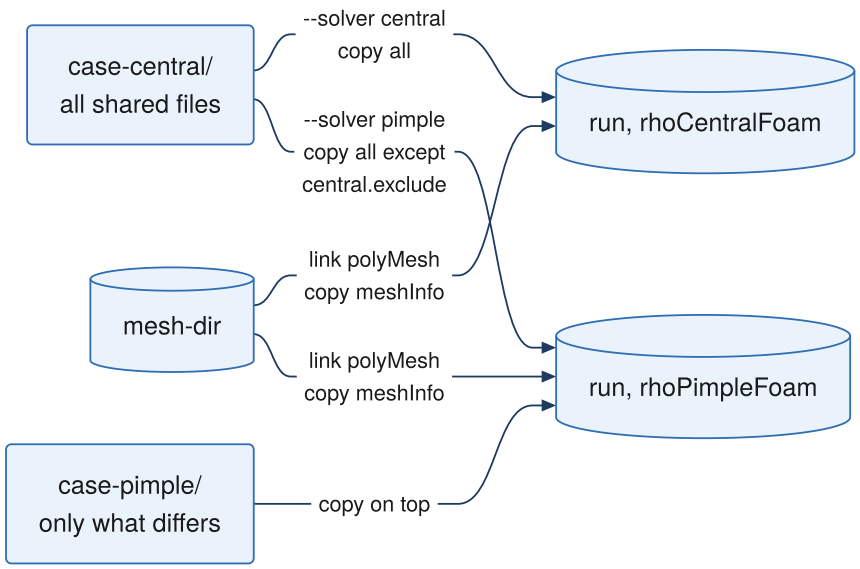


Figure 5.1: How `newCase.sh` assembles a case from the templates and a mesh.

1. It stops if `--mesh` is missing, if the mesh has no `constant/polyMesh`, or if `<run-dir>` exists.
2. It copies `case-central/` into `<run-dir>`, without the mesh, the logs, `0/`, `processor*`, `postProcessing/` and the time directories. With `--solver pimple`, it leaves out the files in `case-pimple/central.exclude` and then copies `case-pimple/` on top (Figure 5.1).
3. It links `constant/polyMesh` to the mesh. With `--refine` it copies it.
4. It copies `<mesh-dir>/meshInfo` to `constant/meshInfo`.
5. It copies `common/Allrun`, `common/Allrefine` and `common/machineLog.py` into the case.
6. It writes `run.info` with the case name, the date, the commit of the repository, the mesh, the number of cells and the options.
7. It renders each `*.j2` file with the values in `config.json` (Section 7.5.3), and removes the `*.j2` file.
8. It writes each flow option into `system/flowConditions`. It stops if the entry does not exist.
9. It writes `--np` into `decomposeParDict` and `--socket` into the file `socket`. With `--init`, it writes the path of the other run into `initFrom` and into `run.info`.
10. With `--refine`, it runs `Allrefine`.
11. It adds to `run.info` the fingerprint of the mesh (`meshHash`) and the physical definition of the case (block `flow`, Section 9.1).

5.2 Run a case

5.2.1 Command

```
cd <run-dir>
./Allrun        # parallel, numberOfSubdomains from system/decomposeParDict
./Allrun 8      # parallel on 8 processes; writes 8 into decomposeParDict
./Allrun 1      # serial
```

5.2.2 Steps that Allrun does

Figure 5.2 shows the steps.

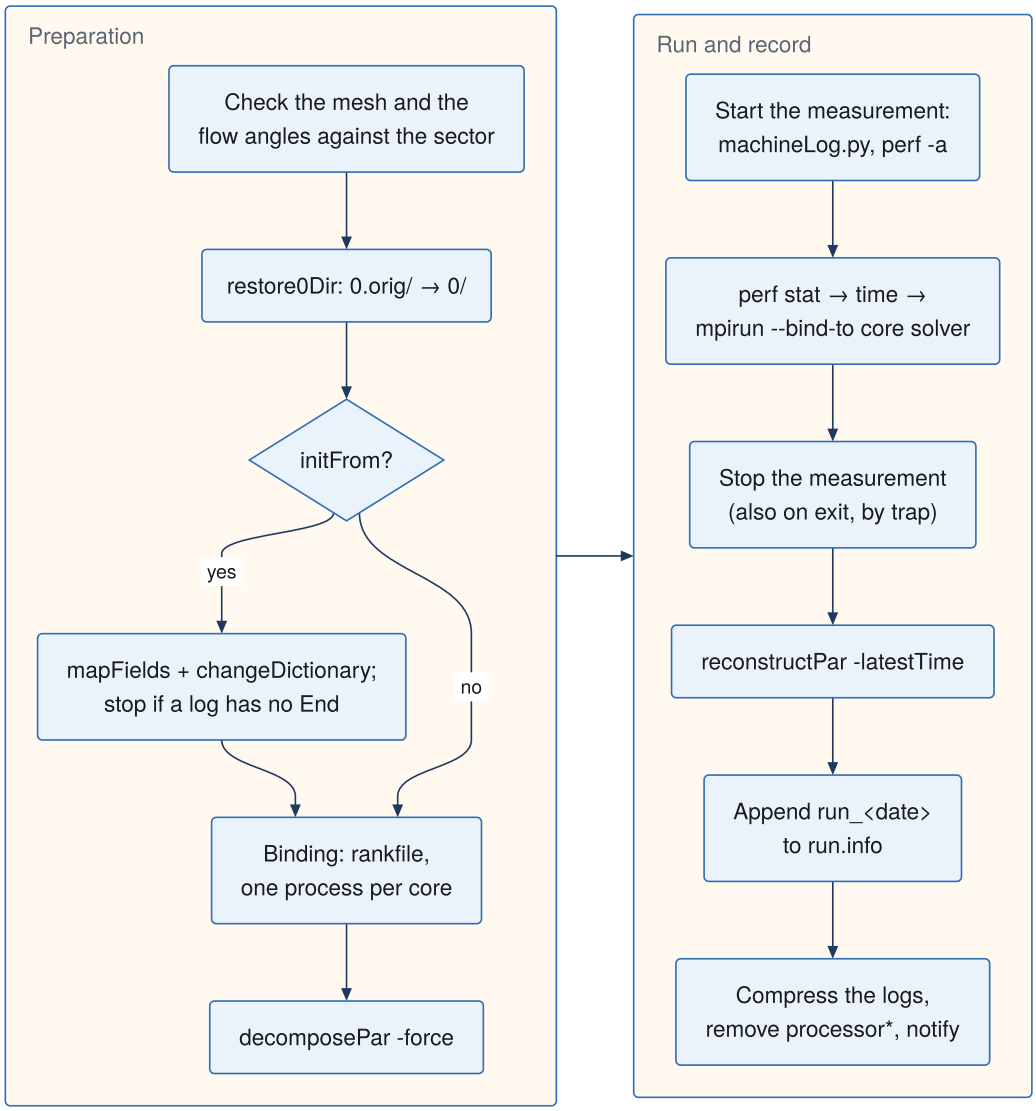


Figure 5.2: Steps of `Allrun`.

1. It stops if `constant/polyMesh` does not exist.
2. It compares `alpha` and `beta` in `flowConditions` with `sector` in `meshInfo` (Table 5.2). It stops if the mesh cannot show the flow angle.
3. It copies `0.orig/` to `0/`. If `initFrom` exists, it maps the other run onto `0/` (Section 5.3).

4. It sets the binding of the processes (Section 5.2.3).
5. It runs `decomposePar -force`.
6. It starts the measurement (Section 8.3) and runs the solver with `mpirun`. It stops the measurement when the solver ends. A trap also stops it if `Allrun` stops early.
7. It runs `reconstructPar -latestTime`, and removes `processor*` if that step ends correctly.
8. It adds a record `run_<date>` to `run.info` (Section 8.2), compresses the logs with `xz` and sends the notification (Section 8.7).

Table 5.2: Flow angles that each mesh sector permits.

`sector`	Permitted angles
`quarter`	$\alpha = 0$ and $\beta = 0$
`half`	$\beta = 0$ (the default mesh)
`full`	All angles

5.2.3 Process binding

`Allrun` binds each MPI process to one physical core when:

- the case has a file `socket` with a socket number, or
- the computer has one socket and the number of processes is not more than the number of physical cores.

It writes a `rankfile` with process i on core i of the socket, and runs `mpirun --bind-to core`. It stops if the processes are more than the cores of the socket. To run without binding, for example a short test beside a sweep, write `none` in the file `socket`.

CAUTION

`Allrun` does not use the OpenFOAM function `runParallel`, because it always adds `--oversubscribe`. With that option, Open MPI 5 ignores the `rankfile`, and each process can move to all CPUs. `run.info` then shows a binding that did not occur. To check the binding of a run, examine each process: `taskset -cp <pid>` must give the two threads of one core.

5.2.4 End of a run

The run stops at `endTime`, or when the stop criterion finds that the mean of C_D is constant (Section 7.5.4). The record in `run.info` gives `status ok` if the solver log ends with `End`.

5.2.5 Continue a run

`controlDict` has `startFrom latestTime`. Run `./Allrun` again in the same directory to continue from the last time directory. The continuation adds a new record to `run.info` and writes the function objects into a new directory with its start time.

CAUTION

`Allrun` runs `decomposePar -force`, which removes the `processor*` directories. If the solver stopped before `reconstructPar`, run `reconstructPar -latestTime` before you run `./Allrun` again. If you do not, you lose the data after the last reconstructed time.

On the first continuation, read the start time in the solver log. Make sure that it is the last time and not 0.

5.2.6 Start a run again from zero

1. Remove the time directories and the processor directories: `rm -rf [1-9]* processor*`.
2. Run `./Allrun`.

`./Allclean` also removes `constant/polyMesh`. After `./Allclean`, make a new case with `newCase.sh`.

5.3 Start from another run

`newCase.sh --init <run-dir>` starts the case from the solution of another run instead of the uniform free stream. The sweep uses it from a coarse mesh to the next finer mesh at the same flight condition, as the OpenFOAM ToolChain does [3, §5.4]. The fine mesh starts with the shock, the wake and the boundary layer in place. The two runs can use different solvers. After `restore0Dir`, `Allrun` does these steps:

1. It reads the value of each `fixedValue` patch in `0/`. These are the values of this run: in a supersonic case, `p` on `inlet`.
2. It runs `mapFields <run-dir> -consistent -sourceTime latestTime`. This maps the internal field and the patch values. The other run can have another mesh of the same domain and patches.
3. It runs `changeDictionary` to write back the values of step 1.
4. It stops if the log of `mapFields` or of `changeDictionary` does not end with `End`.

The free-stream entries of the patches (`freestreamValue`, `inletValue`, `fieldInf`) are not mapped. They keep the values of this run. The other run must have a time directory with the solution.

CAUTION

`mapFields` stops with a segmentation fault when it maps onto the same mesh ($h0 \rightarrow h0$), at the field `Ma` (2026-10-17). The supersonic chain does this mapping (pimple $\rightarrow$ central on $h0$). `Allrun` now stops the run. Before this check, the run continued from the uniform free stream and `Allrun` reported the mapping as correct. See Section 12.3.

5.4 Refine the mesh near the fins (optional)

```
./Allrefine          # same as ./Allrefine tip
./Allrefine tip      # cell set finTip: a band around the fin tip
./Allrefine fins     # cell set finBox: the full fin, from root to tip
```

Do this procedure in the case directory, before `./Allrun`, or give `--refine` to `newCase.sh`. `Allrefine` does these steps:

1. It stops if `0/U` exists. `refineMesh` maps the fields, so a refinement after a run changes the results.
2. It stops if `meshInfo` does not have `fins true` and `finTipR`.
3. It replaces the link to the mesh with a copy, so that it does not change the shared mesh.
4. It runs `topoSet`. `system/topoSetDict` makes the cell sets from the fin geometry in `meshInfo`.
5. It runs `refineMesh`. Each cell in the set becomes 8 cells.
6. It runs `checkMesh` and `renumberMesh`, and writes the new number of cells into `meshInfo`.

Set	Region	Effect on y^+
`finTip`	Annulus from `finTipR` $-$ `tipBand` to `finTipR` $+$ `margin`	None. The region does not touch the body.
`finBox`	Cylinder of radius `finTipR` $+$ `margin`	The first body cell below the fins is half as thick, so y^+ there is approximately half.

Chapter 6

Sweeps

A sweep is two steps. `runMesh.sh` makes the meshes. `runCase.sh` makes and runs the cases on meshes that exist.

6.1 Make the meshes

`meshes.txt` has one mesh on each line: a name, then the options of `Allmesh`. Lines that start with `#`, and empty lines, have no effect.

```
h0  --maxCellSize 2.2627 --nLayers 12 --thicknessRatio 1.1878
h1  --maxCellSize 1.6    --nLayers 10 --thicknessRatio 1.2
h2  --maxCellSize 1.1314 --nLayers 8  --thicknessRatio 1.229
h3  --maxCellSize 0.8    --nLayers 7  --thicknessRatio 1.2
```

```
./runMesh.sh [meshes-file] [meshes-dir]      # default: meshes.txt, □/meshes
```

For each line, `runMesh.sh` does these steps:

1. If the mesh exists, it compares `maxCellSize`, the level of `fins`, `nLayers` and `thicknessRatio` with the line. It keeps the mesh if they are equal, and stops if they are different. It does not compare the geometry: after a change of the STL file, remove the old meshes.
2. If the mesh directory exists but has no mesh, it stops.
3. If not, it runs `Allmesh` with the options of the line. It stops if `Allmesh` stops with an error.

To make only one mesh of the file, give a file with that line only.

6.2 Run the cases

A sweep file has one case on each line: a case name, then the options of `newCase.sh` (Table 5.1).

```
m08-a00-h0  --np 8 --mesh h0 --solver pimple --alpha 0 --Minf 0.8 --pInf 47509 --
    Tinf 266.17
m08-a00-h1  --np 8 --mesh h1 --solver pimple --alpha 0 --Minf 0.8 --pInf 47509 --
    Tinf 266.17 --init m08-a00-h0
```

- Lines that start with `#`, and empty lines, have no effect.
- `--mesh` is mandatory. A name without `/` is a directory in `<meshes-dir>`.

- `--init` is optional. A name without `/` is a directory in `<runs-dir>`. That run must come earlier in the file, because the cases run in sequence.

```
./runCase.sh <sweep-file> [runs-dir] [meshes-dir]   # default: /runs, /meshes
```

For each line, `runCase.sh` does these steps:

1. If `<runs-dir>/<name>` exists, it goes to the next line. If you run `runCase.sh` again after a stop, it thus continues from the first case that does not exist.
2. If the mesh does not exist, it stops.
3. It runs `newCase.sh`. It stops if `newCase.sh` stops with an error.
4. It runs `./Allrun`. If `Allrun` stops with an error, it shows a message and goes to the next line.

6.2.1 Start a sweep

A sweep runs for days. Start it so that it continues when the terminal closes:

1. Do the steps of Section 2.3.
2. Start the sweep:

   ```
   nohup ./runCase.sh sweepSubsonic.nemepad.txt > /log.sweepSubsonic.nemepad 2>&1 &
   ```

3. Monitor it:

   ```
   tail -f /log.sweepSubsonic.nemepad
   ```

4. On the first case, examine the binding (Section 5.2.3) and the measurement files (Section 8.3).

`nohup` keeps `runCase.sh` alive when the terminal closes or the ssh session stops. `tmux` or `screen` do the same.

6.2.2 Stop a sweep

1. Stop `runCase.sh`. No new case starts. The solver of the current case continues until it ends.

   ```
   pkill -f 'runCase.sh sweepSubsonic'
   ```

2. To stop the solver also, stop its `prterun` (Open MPI 5) or `mpirun` (Open MPI 4) by its process number. `Allrun` then reconstructs the last time, writes the record and stops the measurement.

CAUTION

`pkill -f <pattern>` also stops the shell that runs the command if its command line contains the pattern, for example in a script. Find the process number with `ps` and stop it with `kill <pid>`.

CAUTION

`runCase.sh` reads the sweep file line by line while it runs. Do not edit the file in place until the sweep ends: the next line is read from the old position in the new contents. Stop `runCase.sh`, edit the file, and start it again. It skips the cases that exist.

6.3 The sweep of record

Table 6.1 gives the sweep of record.

Table 6.1: The sweep of record.

Item	Value
Mach numbers	0.3, 0.6, 0.8 (`rhoPimpleFoam`); 1.2, 1.8 (`rhoCentralFoam`)
Angles of attack	$0°$, $4°$, $8°$
Meshes	$h0$ (start only), $h1$, $h2$, $h3$ (Table 4.6)
Runs	66: $5 \times 3 \times 4$, plus 6 `rhoPimpleFoam` start runs for $M > 1$
Files	`sweepSubsonic.txt` (36 runs), `sweepSupersonic.txt` (30 runs); `*.nemepad.txt` for `nemepad`

Each Mach number and angle is a chain of runs. Each run starts from the run on the coarser mesh (Figure 6.1). The runs are in the sequence of the meshes, so all the $h0$ results come first. $h0$ is not part of the mesh study.

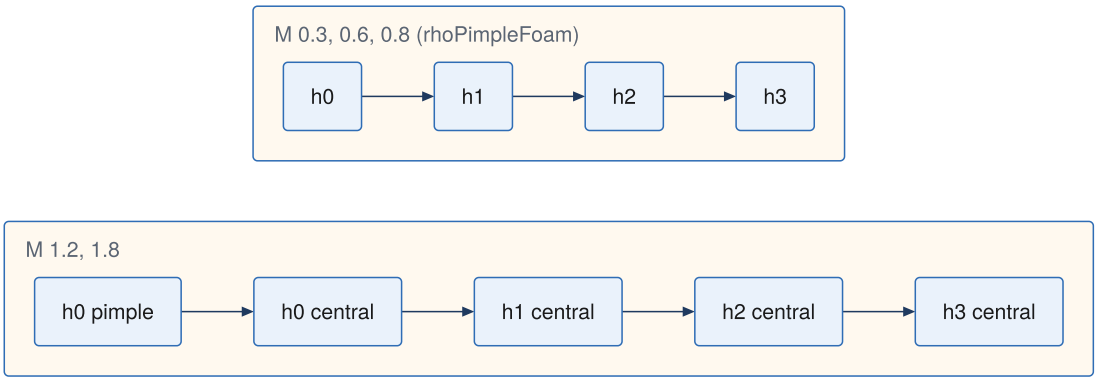


Figure 6.1: Chains of runs. Each arrow is an `--init`.

The supersonic chains start with `rhoPimpleFoam` on $h0$. `rhoCentralFoam` from the uniform free stream stops at the base (Section 7.15).

`--alpha` turns the free stream in the x–y plane, which is the plane of the half mesh. The mesh is the same for all angles. The angle $16°$ was in the sweep until 2026-10-17. It was replaced by $4°$, because the flight had $\alpha < 0.2°$ in the coast (Section 10.4), and at $16°$ the wake left the refinement regions 0.4 m behind the base.

6.3.1 Computers

On `newton`, run the two files at the same time, one on each socket (`--socket 0` and `--socket 1`, 14 processes each). Each chain of `--init` must stay in one file.

On `nemepad` (one socket, 8 cores), use `sweepSubsonic.nemepad.txt` and `sweepSupersonic.nemepad.txt`: the same runs with `--np 8` and no `--socket`. Run one file, then the other. Never run the two at the same time. On $h0$, `m08-a00-h0` takes approximately 2.7 s for each step on `nemepad`, against 1.77 s on one socket of `newton`.

CAUTION

$h3$ does not fit in the memory of `nemepad`. The $h3$ lines of the `nemepad` files are comments until `newton` is back in service. Make only the meshes that you need: `runCase.sh` stops at the first case whose mesh does not exist.

6.4 Flight conditions

The flight conditions are those of the coast phase ($t > 4\,s$) of the IREC flight in Midland, with the atmosphere of that day. `flightData/cdFlight.py` calculates them from the TeleMega data (Chapter 10):

- The pressure at the pad (896 m MSL) is the mean before launch, 90 565 Pa.
- The temperature is 305.0 K at the pad, with a lapse rate of 7.2 K/km: approximately ISA + 20 K.
- The Mach number comes from the integrated speed and that temperature. p_∞ and T_∞ come from the integrated height.

Table 6.2: Flight conditions of the sweep.

M	t [s]	Altitude MSL [m]	`pInf` [Pa]	`Tinf` [K]
1.8	4.1	2171	78 354	295.82
1.2	8.5	4347	60 547	280.15
0.8	14.6	6289	47 509	266.17
0.6	19.8	7437	40 911	257.90
0.3	29.0	8679	34 613	248.96

NOTE

Until 2026-10-17 the sweep used the ISA values at the altitude of the TeleMega (for example $M = 0.8$: 49 800 Pa, 251.72 K). The runs in `~/runs.sin-medicion` use those values.

6.5 Do the sweep again on another computer

1. Do the steps of Chapter 2.
2. Optional: write the geometry again and compare it with the file in the repository. The two files must be equal.

```
python3 mesh/makeRocket.py /tmp/check.stl && cmp /tmp/check.stl mesh/stl/
    Aconcagua_biconvex117.stl
```

3. Make the meshes with `./runMesh.sh`. They must agree with Table 4.10.

4. Start the sweep files for the computer (Section 6.2).

6.6 Typical studies

6.6.1 Mach sweep

1. Write one line for each Mach number in a sweep file.

2. Run `./runCase.sh <sweep-file>`.

3. Measure y^+ on each case. The mesh does not set y^+, and the speed changes by a factor of 6 from $M = 0.3$ to $M = 1.8$.

6.6.2 Mesh convergence

1. Change only `maxCellSize`, with a constant ratio. Do not change the levels. All cell sizes then change by the same ratio.

2. Use the ratio $\sqrt{2}$. The method of Celik et al. [4] needs a ratio larger than 1.3. The mesh does not refine uniformly: the number of cells N grows $\times 1.95$ from $h1$ to $h2$ and $\times 2.34$ from $h2$ to $h3$. The effective ratio $r = (N_{\text{fine}}/N_{\text{coarse}})^{1/3}$ is thus 1.25 and 1.33. Use these values, not $\sqrt{2}$, in Section 11.4.

3. Keep the first cell equal on all meshes (Table 4.6). If t_1 changes, y^+ changes, and the study mixes the wall treatment with the discretization error.

4. Use the same flight condition on all meshes.

6.6.3 Angle-of-attack sweep

```
for a in 0 2 4 6 8; do
    ./newCase.sh □/runs/a$a --mesh □/meshes/h1 --solver pimple --Minf 0.8 --alpha $a
     --np 8
    ( cd □/runs/a$a && ./Allrun 8 )
done
```

6.6.4 Finer fins

Raise the levels of `fins`, `finEdges` and `finRoots` in `meshDict` and make a new mesh, or run `./Allrefine tip` in the case before `./Allrun`.

6.7 Rules

- Do not put meshes in the repository. `Allmesh` makes them again. Put the STL file and `meshDict` in the repository.
- When a change to `meshDict` changes the mesh, commit it with the number of cells and the `checkMesh` output of the new mesh.
- Put the run directories outside the repository and outside `/mnt/c`.

Chapter 7

Solvers and models

This chapter gives the dictionaries of the two templates, the models that they select and the references. Use it to calculate the values of a case by hand and to compare them with the case.

7.1 Summary

Table 7.1: Summary of the templates.

Item	Value
Templates	`case-central/` ($M > 1$, started from pimple, Section 7.15); `case-pimple/` ($M < 1$, Section 7.14)
Solvers	`rhoCentralFoam`; `rhoPimpleFoam`
Formulation	Compressible. Central: density-based, explicit. Pimple: pressure-based, `transonic`
Time	`localEuler` in both; time counts iterations (Section 7.5.1). Central: Courant ramp
Fields in `0.orig/`	`U`, `p`, `T`, `k`, `omega`, `nut`, `alphat`
Turbulence	k–ω SST with wall functions
Gas	Air, perfect gas, constant c_p, Sutherland viscosity
Input	`Minf`, `pInf`, `Tinf`, `alpha`, `beta`, `Ti`, `nuRatio`
Initial condition	Uniform free stream, or another run (Section 5.3)

7.1.1 Why two solvers

The central-upwind flux of `rhoCentralFoam` scales its numerical dissipation with the speed of sound, not with the flow speed. It is thus too dissipative at low Mach numbers. The OpenFOAM ToolChain report [3, §6] measured an error of approximately 100 % in C_A against wind-tunnel data with `rhoCentralFoam` at low Mach numbers, and better results with `rhoPimpleFoam` for $M < 0.8$. Above $M = 1.5$, `rhoPimpleFoam` gave nonphysical results at the nose tip.

The cost is a change of discretization at $M = 1$. A step in the curve $C_D(M)$ there is not separable from the physics. To measure the step, run one Mach number with the two solvers on the same mesh (Section 7.14.4).

The models, the gas, the boundary conditions, the function objects and the flight condition are the same files in the two templates.

7.2 Files of the template

This section lists `case-central/`. `case-pimple/` holds only the files that differ (Section 7.14).

Table 7.2: Files of `case-central/`.

File	Edit	Function
`system/flowConditions`	yes	Flight condition (Section 7.3).
`config.json`	yes	Courant ramp and stop criterion (Section 7.5.3).
`system/flowDerived`	no	Values calculated from `flowConditions`.
`0.orig/*`	no	Initial and boundary conditions (Section 7.9).
`constant/thermophysicalProperties`	no	Gas model (Section 7.7).
`constant/turbulenceProperties`	no	Turbulence model (Section 7.6).
`constant/meshInfo`	no	`newCase.sh` replaces it (Section 4.6).
`system/controlDictBase.j2`	with care	Time control and function objects.
`system/FOs/FOcommon`	with care	Function objects of the two templates, except `residuals` (Section 7.10).
`system/controlDict`	no	Includes `controlDictBase` and adds `maxCo 0.01`.
`system/fileUpdaterControlDict/`, `system/fileUpdater.j2`	no	Steps and schedule of the Courant ramp.
`system/FOs/FOrunTimeControl.j2`	no	Stop criterion.
`system/fvSchemes`, `fvSchemesUpwind`	no	First-order schemes for the start.
`system/fvSchemesVanLeer`	with care	Second-order schemes (Section 7.11).
`system/fvSolution`	no	Linear solvers (Section 7.12).
`system/decomposeParDict`	with `--np`	Parallel decomposition: `method scotch`.
`system/topoSetDict`, `refineMeshDict`	`margin`, `wake`, `tipBand`	Regions of `Allrefine`.
`Allclean`	no	Removes the mesh, the fields and the logs.

CAUTION

Use `foamDictionary -set` only on `system/flowConditions`. On a file with `#include` or `#eval`, `foamDictionary -set` writes the expanded values. The file then does not follow `flowConditions`.

7.3 Flight condition

7.3.1 flowConditions

Table 7.3: Entries of `flowConditions` and the values of the template.

Entry	Default	Unit	Function
`Minf`	1.80	–	Free-stream Mach number.
`alpha`	0	deg	Angle of attack. The velocity turns from $+x$ to $+y$.
`beta`	0	deg	Angle of sideslip. The velocity turns to $+z$.
`pInf`	101 325	Pa	Static pressure (ISA sea level).
`Tinf`	288.15	K	Static temperature (ISA sea level).
`Ti`	0.005	–	Turbulence intensity.
`nuRatio`	5	–	Ratio ν_t/ν in the free stream.

The sweep gives `Minf`, `pInf`, `Tinf` and `alpha` for each case (Table 6.2). To change a value, use `newCase.sh`, or:

```
foamDictionary system/flowConditions -entry Minf -set 1.5
```

7.3.2 flowDerived

`flowDerived` includes `flowConditions` and calculates the values below with `#eval`. The fields in `0.orig/` and `controlDictBase` include `flowDerived`. A change to `flowConditions` thus changes all the conditions.

Table 7.4: Values of `flowDerived` at two sweep conditions (`Ti` $= 0.005$, `nuRatio` $= 5$).

Value	Formula	$M = 0.8$	$M = 1.8$
`aInf` $[\mathrm{m/s}]$	$\sqrt{\gamma RT}$	327.06	344.79
`Uinf` $[\mathrm{m/s}]$	Ma	261.64	620.62
`rhoInf` $[\mathrm{kg/m^3}]$	$p/(RT)$	0.6218	0.9227
`muInf` $[\mathrm{Pa\,s}]$	$A_s\sqrt{T}/(1 + T_s/T)$	1.681×10^{-5}	1.828×10^{-5}
`kInlet` $[\mathrm{m^2/s^2}]$	$1.5\,(U\,\mathtt{Ti})^2$	2.567	14.44
`nutInlet` $[\mathrm{m^2/s}]$	`nuRatio` ν	1.352×10^{-4}	9.903×10^{-5}
`omegaInlet` $[1/\mathrm{s}]$	k/ν_t	1.900×10^{4}	1.459×10^{5}

Value	Formula
`Ux`, `Uy`, `Uz`	$U\,(\cos\alpha\cos\beta,\ \sin\alpha\cos\beta,\ \sin\beta)$
Drag direction `dx`, `dy`, `dz`	$(\cos\alpha\cos\beta,\ \sin\alpha\cos\beta,\ \sin\beta)$
Lift direction `lx`, `ly`, `lz`	$(-\sin\alpha,\ \cos\alpha,\ 0)$

Constants in `flowDerived`: `Rgas 287.05`, `gammaGas 1.4`, `Prt 0.85`. They must agree with `thermophysicalProperties` (Section 7.7). The Sutherland constants are in the formula of `muInf`.

7.4 Equations and numerical method of rhoCentralFoam

`rhoCentralFoam` does not solve a pressure equation. It advances the conserved variables ρ, $\rho\mathbf{U}$ and ρE with a central-upwind flux, then it applies the diffusive terms implicitly. `fvSolution` thus has no `SIMPLE` or `PIMPLE` dictionary and no relaxation factors.

Flux. `fluxScheme Kurganov`. The scheme divides the face flux with the local wave speeds a^+ and a^-. It captures shocks without a Riemann solver and without artificial viscosity [5, 6]. Greenshields et al. [7] give the implementation and validation in OpenFOAM.

Reconstruction. The primitive variables go to the faces with the van Leer limiter: `reconstruct(rho) vanLeer`, `reconstruct(U) vanLeerV`, `reconstruct(T) vanLeer`. At a discontinuity the limiter decreases the scheme to first order. In smooth regions the scheme stays second order [8]. `vanLeerV` limits in the direction of the largest gradient.

7.5 Local time step, Courant ramp and stop criterion

7.5.1 Local time step

The two templates use `ddt localEuler`, as the OpenFOAM ToolChain does [3, §5.3]. Each cell advances with its own time step, the step that gives `maxCo` in that cell. The run is a pseudo-transient march to the steady state: the intermediate states are not physical, the converged state is.

`deltaT 1` in `controlDict`, so the time counts iterations: `endTime`, `writeInterval`, the ramp and the stop criterion are all in iterations.

On $h1$ at $M = 1.8$ the local step goes from 1.5×10^{-10} s in the smallest cells to 1.6×10^{-5} s in the far field. With a global `Euler` step, all the domain advanced at 1.5×10^{-10} s: approximately 6.8×10^5 steps for each flow-through time.

Entry	Central: read from	Pimple: read from	Value
`maxCo`	`controlDict` (the ramp changes it)	`fvSolution PIMPLE`	ramp to `final_Co`; pimple from `config.json`
`maxDeltaT`	`controlDict`	`fvSolution PIMPLE`	1 s: no limit
`rDeltaTSmoothingCoeff`	`controlDict`	`fvSolution PIMPLE`	0.1
`rDeltaTDampingCoeff`	not read	`fvSolution PIMPLE`	pimple 0.9

Source: `setRDeltaT.H` of `rhoCentralFoam` and of `rhoPimpleFoam` in v2412.

CAUTION

With `localEuler`, the line `Mean and max Courant Numbers` of `rhoCentralFoam` uses `deltaT` of `controlDict` (1), not the local step. Values of the order of 10^7 are normal. `rhoPimpleFoam` does not write the Courant number. Look at `Flow time scale min/max`: the minimum is the step of the most restrictive cell. A sudden decrease shows a cell that diverges.

7.5.2 Courant ramp (central only)

A uniform free stream on a no-slip wall causes a large transient. The ramp of the ToolChain [3, §5.4.1] starts with `maxCo 0.01` and first-order reconstruction, and doubles `maxCo` step by step. The function objects `timeActivatedFileUpdate` in `system/fileUpdater` copy a file onto `system/controlDict` or `system/fvSchemes` at each switch iteration. `runTimeModifiable yes` makes the solver read the new file.

Table 7.5: Courant ramp of `case-central/`.

Phase	Iterations	`maxCo`
Upwind	0, 20, 35, 50, 65, 80, 100	0.01, 0.02, 0.04, 0.08, 0.16, 0.32, 0.5
van Leer	`vanLeer` + 0, 20, 35, 150, 300, 800, 2000	0.01, 0.02, 0.04, 0.08, 0.16, 0.32, 0.5
Final	`minIter`, or the end of the ramp + 1 if later	`final_Co`, with the stop criterion

Each `maxCo` is limited to `final_Co`. The final step never comes before the end of the ramp. `rhoCentralFoam` has no damping of the local step, and a jump of `maxCo` from 0.04 to 0.5 gave a floating point exception six iterations later ($h1$, $M = 1.8$). With the default values the ramp ends at iteration 4000.

`rhoPimpleFoam` has no ramp. It was stable from the uniform free stream on $h1$ at $M = 0.3$.

7.5.3 config.json

`newCase.sh` renders all `*.j2` files with these values.

Table 7.6: Keys of `config.json`.

Key	Central	Pimple	Function
`maxCo`	–	0.5	Courant number of each cell.
`vanLeer`	2000	–	Iteration of the change from upwind to van Leer. `-1` keeps upwind.
`minIter`	3000	1000	First iteration of the stop criterion.
`final_Co`	0.5	–	Largest `maxCo` of the ramp.
`endIter`	20000	20000	Last iteration if the criterion does not stop the run.
`coeffs_fields`	`["Cd"]`	`["Cd"]`	Coefficients in the stop criterion.

Key	Central	Pimple	Function
`coeffs_variation`	`[0.007]`	`[0.007]`	Absolute tolerance: 0.007 is 2 % of $C_D = 0.35$.
`coeffs_window`	`[1000]`	`[1000]`	Window of each coefficient, in iterations.

The values come from the ToolChain, which used another rocket and other meshes. Adjust them from the first runs on $h1$.

7.5.4 Stop criterion

File: `system/FOs/FOrunTimeControl.j2`, the same for the two templates. For each field in `coeffs_fields`, the condition `average` calculates a moving mean over `coeffs_window` iterations, from `minIter`. The run stops when, for all the fields, the window is complete and

$$|\overline{C} - C| < \texttt{coeffs_variation}.$$

`endIter` is the second limit.

The sweep stops on C_D only, also at $\alpha = 4°$ and $8°$. Examine that C_L and C_m are also flat in `coefficient.dat`. You can add `Cl` and `CmPitch` to `coeffs_fields`. No tolerance for `CmPitch` is chosen: about the nose tip with $l_{\text{ref}} = 0.151\,m$ it is 10 to 100 times C_L.

To change the tolerance of a running case, edit `tolerance` in its `system/FOs/FOrunTimeControl` and run `touch system/controlDict`. The solver reads the file again and starts the mean from zero. The run thus cannot stop before one more window.

7.6 Turbulence

File: `constant/turbulenceProperties`: `simulationType RAS`, `RASModel kOmegaSST`.

Closure. The Boussinesq hypothesis gives the Reynolds stress from an eddy viscosity and the strain rate:

$$-\overline{u_i' u_j'} = \nu_t \left(\frac{\partial U_i}{\partial x_j} + \frac{\partial U_j}{\partial x_i} \right) - \frac{2}{3} k\, \delta_{ij}, \qquad \nu_t = \frac{a_1 k}{\max(a_1 \omega,\ SF_2)}.$$

The denominator is the shear-stress limiter. It limits the production of ν_t in adverse pressure gradients, which occur on the boattail and at the fin-body junction.

References. Menter [9]; Menter et al. [10], the version that OpenFOAM uses. When you compare with the literature, find which version the source uses. Default constants (`src/TurbulenceModels/.../kOmegaSST/`): $\alpha_{k1} = 0.85$, $\alpha_{k2} = 1.0$, $\alpha_{\omega1} = 0.5$, $\alpha_{\omega2} = 0.856$, $\beta_1 = 0.075$, $\beta_2 = 0.0828$, $\beta^* = 0.09$, $\gamma_1 = 5/9$, $\gamma_2 = 0.44$, $a_1 = 0.31$, $b_1 = 1.0$, $c_1 = 10$, F_3 off.

7.6.1 Wall treatment

On `cone`, `walls`, `tail` and `fins`:

Field	Condition	Reason
`nut`	`nutUSpaldingWallFunction`	Spalding's law is continuous in y^+ [11]. The first cell can be in the viscous sublayer, the buffer layer or the log layer.
`k`	`kLowReWallFunction`	Also continuous in y^+.
`omega`	`omegaWallFunction`	Blends the sublayer and log-layer values of ω.
`alphat`	`compressible::` `alphatWallFunction`, `Prt 0.85`	Turbulent thermal diffusivity from ν_t.

These conditions need wall patches of type `wall`. The mesh does not set y^+. Measure y^+ on each case.

7.7 Thermophysical properties

File: `constant/thermophysicalProperties`.

Entry	Value	Function
`type`	`hePsiThermo`	Energy-based, $\psi = 1/(RT)$. `rhoCentralFoam` accepts only `psiThermo`.
`mixture`	`pureMixture`	One species.
`transport`	`sutherland`	$\mu(T) = A_s\sqrt{T}/(1 + T_s/T)$.
`thermo`	`hConst`	Constant c_p.
`equationOfState`	`perfectGas`	$p = \rho RT$.
`energy`	`sensibleInternalEnergy`	The solver solves e. `rhoCentralFoam` needs it.

Table 7.7: Constants of the gas.

Constant	Value	Calculation
`molWeight`	28.96 kg/kmol	$R = 8314.46/28.96 = 287.10\,J/(kg\,K)$
`Cp`	1005 J/(kg K)	$\gamma = c_p/(c_p - R) = 1.400$
`As`	1.4792×10^{-6}	At 288.15 K: $\mu = 1.790 \times 10^{-5}\,Pa\,s$
`Ts`	116 K	
Pr	0.690	From the Eucken correction (below), not an input
`Prt`	0.85	Turbulent Prandtl number for air [12]

White [13, eq. 1-36] gives Sutherland's law as $\mu = \mu_{\text{ref}}(T/T_{\text{ref}})^{1.5}(T_{\text{ref}} + S)/(T + S)$. The OpenFOAM form $A_s\sqrt{T}/(1 + T_s/T) = A_s T^{1.5}/(T + T_s)$ is the same equation.

Conductivity and Prandtl number. With `sutherland`, OpenFOAM calculates the conductivity with the modified Eucken correction, $\kappa = \mu\, c_v\,(1.32 + 1.77\,R/c_v)$. The laminar Prandtl number is thus

$$Pr = \frac{c_p}{c_v\,(1.32 + 1.77\,R/c_v)} = 0.690,$$

not 0.72. Another solver must use the same value (Chapter 9).

Gas constant. `flowDerived` uses $R = 287.05$ and the thermophysical model $8314.46/28.96 = 287.10$. The difference in `rhoInf` is 0.02 %.

Limit of hConst. The stagnation temperature is $T_0 = T\,(1 + \frac{\gamma-1}{2} M^2)$. At the flight conditions (Table 6.2), T_0 is 300 K at $M = 0.8$ and 487 K at $M = 1.8$. The error of a constant c_p is less than 0.5 % below $M = 0.8$, approximately 2 % at $M = 1.8$ and 4 % at $M = 2.5$. Above $M = 2.5$, replace `hConst` with `janaf`. The coefficients for air are in `$FOAM_TUTORIALS/compressible/rhoCentralFoam/biconic25-55Run35`.

7.8 Temperature limits

`case-central/` has none. `rhoCentralFoam` in v2412 does not read `fvOptions`, so a `limitTemperature` entry has no effect. `case-pimple/` has one (Section 7.14). With `perfectGas`, a negative T gives an imaginary speed of sound, and the solver stops with a floating point exception. The Courant ramp is the only protection of the central template.

7.9 Boundary conditions

The number of characteristics that go in and go out on each boundary sets the conditions. `U` and `T` use the same condition at all Mach numbers. `p` does not: `0.orig/p` selects the branch with `#if ${{ $Minf > 1 }}`.

Field	`inlet`, `outlet`, `box`	Function
`U`	`freestreamVelocity`	Fixed where the flow goes in, extrapolated where it goes out.
`T`, `k`, `omega`	`inletOutlet`	The same.
`nut`, `alphat`	`calculated`	From the turbulence model.

p, $M \leq 1$. `freestreamPressure` on `inlet`, `outlet` and `box`. It extrapolates where the flow goes in and is fixed where the flow goes out. In subsonic flow, one acoustic wave goes in through the outlet.

p, $M > 1$. Each boundary has its own condition:

Boundary	Condition	Reason
`inlet`	`fixedValue`	All characteristics go in.
`outlet`	`zeroGradient`	All characteristics go out.
`box`	`waveTransmissive`	The normal velocity is approximately 0. Waves cross the boundary in the two directions. The condition moves p out at $u + c$ and relaxes it to `fieldInf` over `lInf` = 10 m.

In v2412, `freestreamPressure` has no supersonic mode: on a supersonic outlet it fixes p. This is the reason for the branch. `waveTransmissive` is the non-reflecting condition of Poinsot and Lele [14]. At $M = 1.8$ the Mach cone crosses `box` (Section 4.4). A `fixedValue` there reflects the wave into the domain.

Walls. `U`: `noSlip`. `p`: `zeroGradient`. `T`: `zeroGradient` (adiabatic). At $M = 1.8$ the recovery temperature is approximately 470 K. In a motor burn of 4 s, the airframe does not get to this value. Zero heat flux is thus the correct limit.

The `symm` entries in `0.orig/` apply only to `half` and `quarter` meshes.

7.10 Function objects

File: `system/FOs/FOcommon`, included by the two templates. `residuals` is in `controlDictBase.j2` and in `case-pimple/system/controlDict.j2`, because the solved fields differ.

Table 7.8: Function objects.

Name	Type	Output	Interval
`forceCoeffs1`	`forceCoeffs`	C_D, C_L, C_m and others	50 steps
`forces1`	`forces`	Forces and moments, pressure and viscous parts	50 steps
`residuals`	`solverInfo`	Residuals (and `p` in pimple)	50 steps
`timeInfo`	`timeInfo`	CPU and wall time	50 steps
`MachNo`, `yPlus`, `wallShearStress`, `Cp`		Fields	write time
`surfaces`	`surfaces`, VTK	p, U, T, ρ, M, C_p, τ_w, y^+ on `symm` and on the walls	end of the run
`samples`	`sets`, CSV	Lines on the skin, in the wake, on the axis, and a free-stream point (Table 8.2)	end of the run

Reference	Value	Source
`patches`	`$wallPatches`	`constant/meshInfo`
`rho`	`rho`	The density field: p is in Pa.
`rhoInf`, `magUInf`	`$rhoInf`, `$Uinf`	`flowDerived`. For the dynamic pressure.
`Aref`, `lRef`	`$Aref`, `$lRef`	`constant/meshInfo`
`CofR`	`(0 0 0)`	The nose tip.
`pitchAxis`	`(0 0 1)`	z axis
`dragDir`, `liftDir`	from α and β	Wind axes (`flowDerived`).

Time control: `startFrom latestTime`, `endTime` = `endIter`, `deltaT 1`, `writeInterval` 500 steps, `purgeWrite 1` (only the last time directory stays), `writeFormat binary`.

7.11 Discretization schemes

`system/fvSchemesVanLeer` contains the full set. `system/fvSchemes` and `system/fvSchemesUpwind` include it and set the three `reconstruct` entries to `upwind`. The ramp copies `fvSchemesUpwind` at the start and `fvSchemesVanLeer` at iteration `vanLeer`. Do not edit `system/fvSchemes`.

Term	Scheme	Reason
`fluxScheme`	`Kurganov`	Central-upwind flux.
`reconstruct(rho)`, `reconstruct(T)`	`vanLeer`	TVD.
`reconstruct(U)`	`vanLeerV`	TVD, vector.
`ddt`	`localEuler`	Local time step.
`grad`	`Gauss linear`	
`div(tauMC)`	`Gauss linear`	Deviatoric viscous stress.
`div(phi,k)`, `div(phi,omega)`	`Gauss upwind`	k and ω must stay positive.
`laplacian`, `snGrad`	`limited corrected 0.33`	Non-orthogonal correction, limited.
`wallDist`	`meshWave`	Wall distance for the SST model.

7.12 Linear solvers

Fields	Solver	Tolerance	`relTol`
`rho`, `rhoU`, `rhoE`	`diagonal`	–	–
`U`, `e`	`smoothSolver`, `symGaussSeidel`, 2 sweeps	10^{-9}	0.01
`k`, `omega`	`smoothSolver`, `symGaussSeidel`, 1 sweep	10^{-9}	0.1

The conserved variables are explicit. Only the implicit diffusive corrections and the turbulence equations use a real linear solver.

7.13 Origin of the configuration

The central template follows `$FOAM_TUTORIALS/compressible/rhoCentralFoam/biconic25-55Run35` and the Courant ramp of the ToolChain. Differences from the tutorial:

Item	Tutorial	Template	Reason
Turbulence	laminar	`kOmegaSST`	The rocket flow is turbulent.
Walls	`maxwellSlipU`, `smoluchowskiJumpT`	`noSlip`, adiabatic	The tutorial is a low-density flow.
`laplacian`, `snGrad`	not limited	`limited corrected 0.33`	Non-orthogonal cells.
`ddt`	`localEuler`	`localEuler`	The template used `Euler` until 2026-10-12, after a misreading of the Courant number (Section 12.4).

7.14 rhoPimpleFoam template

`case-pimple/` holds only the files that differ from `case-central/`. The other files (`0.orig/`, `constant/`, `flowConditions`, `flowDerived`, `decomposeParDict`, `FOs/FOcommon`, `FOs/FOrunTimeControl.j2`) are those of `case-central/`: change them there.

File	Function
`config.json`	`maxCo`, `minIter`, `endIter`, stop criterion.
`system/controlDict.j2`	Time in iterations, function objects, `residuals` with `p`.
`system/fvSchemes`	Schemes (below).
`system/fvSolution.j2`	`PIMPLE` and the local-time-step controls.
`system/fvOptions`	`limitTemperature` from 100 to 1000 K.
`central.exclude`	Files of `case-central/` that this template replaces or does not use.

7.14.1 Schemes

From `templates/rhoPimpleFoamTransonic` of the ToolChain, with the `laplacian` and `snGrad` of the central template.

Term	Scheme	Reason
`ddt`	`localEuler`	Section 7.5.1.
`grad`	`cellLimited Gauss linear 1`; 0.333 for `p` and `e`	Bounded gradients.
`div(phi,U)`	`Gauss vanLeerV`	TVD, second order away from shocks.
`div(phi,e)`, `div(phid,p)`	`Gauss limitedLinear 1`	Bounded. `phid` is the convective flux of p in the `transonic` formulation.
`div(phi,K)`	`Gauss linear`	Kinetic energy.
`div(phiv,p)`	`Gauss upwind`	Pressure work in the e equation.
`div(phi,k)`, `div(phi,omega)`	`Gauss upwind`	As central.

7.14.2 PIMPLE

Entry	Value	Reason
`transonic`	yes	Density-weighted pressure equation for $M = 0.6$ to 0.8. Valid at $M = 0.3$.
`nOuterCorrectors`, `nCorrectors`	1, 2	Pseudo-steady march: one PISO pass for each iteration.
`nNonOrthogonalCorrectors`	1	Maximum non-orthogonality $66°$.
`pMinFactor`, `pMaxFactor`	0.05, 6	Limits of p relative to the boundary values. 6 covers the stagnation pressure behind a normal shock at $M = 1.8$.
`maxCo`, `rDeltaTSmoothingCoeff`, `rDeltaTDampingCoeff`, `maxDeltaT`	0.5, 0.1, 0.9, 1	Section 7.5.1.

Linear solvers: `p` with `GAMG`, tolerance 10^{-8}, `relTol 0.01`; `rho`, `U`, `e`, `k`, `omega` with `smoothSolver` `symGaussSeidel`, `relTol 0.1`.

Temperature limit. `limitTemperature` between 100 and 1000 K. The free stream of the sweep is at 249 to 296 K, and the stagnation temperature at $M = 1.8$ is 487 K. The limit only protects the start. Count the cells that it limits in the log (`LimitedCells`).

7.14.3 Tests on h1 (2026-10-12)

$h1$, half mesh, 8 processes on `nemepad`, ISA conditions.

Case	Result
Pimple, $M = 0.3$, 150 iterations from the uniform free stream	Stable. 4.6 s for each iteration. y^+ mean (max): `cone` 6.6 (8.8), `walls` 3.1 (7.8), `tail` 2.1 (6.1), `fins` 11.1 (30.1).
Pimple, $M = 0.3$, from the case above with `--init`	`mapFields` and the run work. `mapFields` warns about the tet decomposition on approximately 100 layer faces.
Pimple, $M = 0.3$, three sets of schemes	The same within 1 %. The schemes do not change the cold cell (below).
Pimple, $M = 1.8$, 800 iterations	C_D 0.69 (50), 0.54 (300), 0.47 (500 to 800).
Central, $M = 1.8$, from the uniform free stream, three variants	Floating point exception at iteration 74 to 256 (Section 7.15).
Central, $M = 1.8$, `--init` from the pimple run, 400 iterations	Stable. T from 136 to 481 K. C_D 0.51 to 0.53 in van Leer, not converged.

At $M = 0.3$, at iteration 1, `limitTemperature` limits 160 cells, then 0 to 2. The lowest T is in a layer cell near $x = 2.925\,m$, $r = 0.06\,m$, between the fin trailing edge and the boattail, almost at rest. It is the start transient in a region with almost no flow.

7.14.4 Two solvers at the same Mach number

The sweep runs $M = 1.2$ and 1.8 on $h0$ with the two solvers, so it gives the difference between the solvers at those Mach numbers, on the coarse mesh. To measure the step on a mesh of the study, add a pimple case on $h1$ started from the $h0$ pimple run, or a central case at $M = 0.8$.

7.15 Start of the supersonic cases

From the uniform free stream, the flow behind the base starts at U_∞: the base acts as a piston and empties the region behind it. The cells there are thin layer cells with a small local time step, and they fill slowly. On $h1$ at $M = 1.8$ with the pimple template ($p_\infty = 80\,127\,Pa$, ISA at that time):

Table 7.10: Base pressure and temperature during the pimple start, $h1$, $M = 1.8$.

Iteration	p at the base [Pa]	T at the base [K]
305	7 900	166
455	57 000	∼320
755	46 000	350 to 414

At the end $p_{\text{base}}/p_\infty \approx 0.57$, in the range of a power-off base at $M = 1.8$. During the first 300 iterations, T at the base stays at the 100 K limit of `limitTemperature`.

`rhoCentralFoam` reads no `fvOptions`, so it has no such limit. In three tests from the uniform free stream, T at the base fell to 0 and the solver stopped at iterations 74 to 256. The parameters of the local time step did not change this. For $M > 1$ the sweep thus runs the pimple template first on $h0$ and starts the central case from it.

CAUTION

The central run started this way on $h0$ at $M = 1.8$ did not converge: C_D went from 0.44 to 0.89 in 584 iterations and still increased, mostly from the viscous force (Section 12.2). Do not run the supersonic sweep until this defect is corrected.

Chapter 8

Results and measurement

Each run records its coefficients, its samples, its cost and the state of the computer in small files. A run thus can be compared with another solver without the fields of gigabytes.

8.1 Outputs of a run

Table 8.1: Outputs of a run. `<t0>` is the start time: 0 for a new run.

File	Contents
`postProcessing/forceCoeffs1/<t0>/coefficient.dat`	C_D, C_L, `CmPitch`, `CmRoll`, `CmYaw`, C_s, each also split into front `(f)` and rear `(r)`. Every 50 steps.
`postProcessing/forces1/<t0>/force.dat`, `moment.dat`	Force in N and moment in N m: total, pressure and viscous. Every 50 steps.
`postProcessing/residuals/<t0>/solverInfo.dat`	Initial residuals. Every 50 steps.
`postProcessing/yPlus/<t0>/yPlus.dat`	y^+ minimum, maximum and mean for each wall patch, at each write time.
`postProcessing/timeInfo/<t0>/timeInfo.dat`	CPU and wall time, total and for the last 50 steps.
`postProcessing/surfaces/<t>/`	VTK of the skin and the symmetry plane, at the end.
`postProcessing/samples/<t>/*.csv`	Lines and points for the comparison with another solver, at the end (Table 8.2).
Last time directory	`p`, `T`, `U`, `rho`, `k`, `omega`, `nut`, `alphat`, `Ma`, `static(p)_coeff` (C_p), `yPlus`, `wallShearStress`.
`run.info`	Record of the case and of each execution (Section 8.2).
`log.<app>.<date>.xz`	Compressed logs. Read them with `xzless` or `xzgrep`.

File	Contents
`log.perf.<date>.xz`, `log.energy.<date>.xz`, `log.machine.<date>`	Measurement of the computer (Section 8.3).

Open the fields in ParaView with `paraview case.foam`. These quantities are not written, but `postProcess` calculates them from the last time without a new run:

Quantity	Command
Vorticity, Q criterion	`postProcess -func vorticity -latestTime`, `-func Q`
Total pressure and temperature	function object `pressure` with `mode total`; $T_0 = T\,(1 + \frac{\gamma-1}{2} M^2)$

Table 8.2: Samples for the comparison with another solver (function object `samples`).

Set	Contents
`skinZ0`	The wall cut 0.5 mm off the plane $z = 0$. Columns x, y, z: windward is $y < 0$, leeward is $y > 0$.
`skinY0`	The wall cut 0.5 mm off the plane $y = 0$ (the side). The two planes pass between the fins.
`wake05`, `wake1`, `wake2`, `wake5`	Lines in y from -0.6 to 0.6 m on $z = 0$, at 0.5, 1, 2 and 5 diameters behind the base.
`axis`	From $x = -0.5\,m$ to 20 diameters behind the base.
`freeStream`	One point 1 m in front of the nose tip.

The samples contain p, T, U, ρ, M, k, ω, ν_t, C_p, τ_w and y^+, with `cellPoint` interpolation.

8.2 run.info

`newCase.sh` makes `run.info`. `Allrun` adds a record `run_<date>` each time that it runs. The file is an OpenFOAM dictionary: read it with `foamDictionary`.

Table 8.3: Entries of `run.info`.

Entry	Written by	Contents
`case`, `created`, `commit`	`newCase.sh`	Name, date, `git describe --dirty` of the repository.
`mesh`, `nCells`, `options`, `init`	`newCase.sh`	Mesh, number of cells, options, start run.
`meshHash`, `flow`	`newCase.sh`	Fingerprint of the mesh and physical definition of the case (Section 9.1).
`host`, `cpu`, `np`, `binding`	`Allrun`	Computer, processes, binding (Section 5.2.3).
`start`, `end`, `wallSeconds`	`Allrun`	Wall time of decomposition, solver and reconstruction.

Entry	Written by	Contents
`solverWallSeconds`	`Allrun`	Wall time of the solver only.
`status`, `steps`, `simTime`	`Allrun`	`ok` if the solver log ends with `End`; steps; first and last time.
`maxRSSkB`	`Allrun`	Peak memory of the largest MPI process.
`coreUsPerCellStep`	`Allrun`	wall $\cdot\, n_p \cdot 10^6/(N_{\text{cells}} \cdot \text{steps})$.
`cyclesPerCellStep`	`Allrun`	CPU cycles of all processes $/(N_{\text{cells}} \cdot \text{steps})$.
`Cd`	`Allrun`	Last value of `coefficient.dat`.
`machine`, `measured`	`Allrun`	State of the computer and measurement (Table 8.4).

8.3 Measurement of the computer

The wall time of a run depends on the computer as much as on the solver: CPU profile, turbo, temperature, suspension. `Allrun` measures the computer during the solver, without a change to the solver (Figure 8.1).

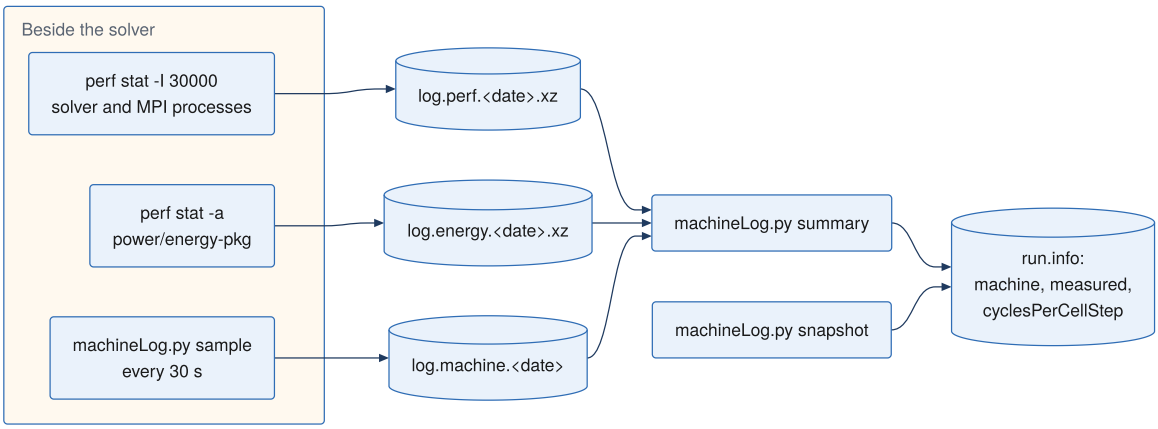


Figure 8.1: Measurement during the solver and its record in `run.info`.

File	Tool	Contents
`log.perf`	`perf stat -I 30000` on the solver	Every 30 s: task clock, cycles, instructions, front-end stalls, cache misses, migrations, context switches.
`log.machine`	`machineLog.py sample`	Every 30 s: wall, monotonic and boot clocks, CPU frequency, load, temperatures of CPU, memory and NVMe, fans, CPU profile.
`log.energy`	`perf stat -a -e power/energy-pkg/`	Energy of the CPU package every 30 s. Only if `perf` can count on the whole system.

Table 8.4: Block `measured` of `run.info`.

Entry	Meaning
`suspendedSeconds`	Change of the boot clock minus change of the monotonic clock: the time that the computer was suspended. If it is not 0, the wall time is not a cost.
`mhzMean`, `mhzMinOfMean`	Mean clock of all logical CPUs, and the lowest mean of 30 s.
`tCpuMax`, `tDimm<i>Max`, `tNvmeMax`	Highest temperatures. DDR5 memory doubles its refresh above 85 °C.
`profiles`	The CPU profiles during the run. If it is not only `performance`, the run does not compare.
`cpuSeconds`	CPU time of all solver processes.
`cycles`, `instructions`	Totals of all processes.
`GHzEffective`	cycles/cpuSeconds: the real clock of the solver, with turbo and throttling.
`IPC`	instructions/cycles. Low ($\lesssim 1$) in a code that the memory bandwidth limits.
`frontendStallFraction`	Cycles with a stopped front end, divided by the cycles.
`cacheMisses`, `cpuMigrations`, `contextSwitches`	Indicators of noise. Many migrations show a binding that does not work.
`energyPkgJ`	Energy of the CPU package during the solver.

The block `machine` gives the state at the start: kernel, sockets, cores, threads for each core, memory, CPU governor and energy preference, boost, maximum frequency, CPU profile, fan level, `perf_event_paranoid`, OpenFOAM build and Open MPI version.

`cyclesPerCellStep` is the cost that does not depend on the clock. It includes the start (the solver reads the mesh), which is small in a run of thousands of steps.

8.3.1 Requirements

- `kernel.perf_event_paranoid` ≤ 1 to count the cycles. If not, `Allrun` shows a message with `!!` and runs without `cycles`.
- ≤ 0 (or `CAP_PERFMON`) to measure the energy (Section 2.3).

8.3.2 Procedure to get a comparable cost

1. Do the steps of Section 2.3.
2. Run one case at a time, with nothing heavy beside it.
3. Use the same number of processes and the same binding for the two solvers: one process for each physical core.
4. Before you use the cost of a run, examine `suspendedSeconds`, `profiles`, `GHzEffective` and `cpuMigrations`.

8.3.3 Do not divide by the clock frequency

A correction of the wall time by the clock supposes that the time changes as $1/f$. This is true for a code that the computation limits, not for a code that the memory bandwidth limits. The two solvers differ in this property: OpenFOAM (low order) depends more on the memory bandwidth, Nektar++ at a high polynomial order more on the clock. To measure it, run the same short case at several clock limits (`scaling_max_freq` at 2, 3 and 4 GHz) and fit

$$t(f) = t_{\mathrm{mem}} + \frac{C}{f}.$$

This procedure is not automated.

8.3.4 Runs without this measurement

The runs before 2026-10-17 have only `timeInfo.dat`. An interval with wall time much larger than CPU time is a suspension. An interval with CPU time much above the median is a low clock or a load beside the run. Without those intervals, the $h0$ runs on `nemepad` cost 20 to 22 core-µs for each cell and step.

8.4 Monitor a run

```
./post/monitor.sh <run-dir> [seconds]     # default 30 s
```

The script plots C_D and C_L against the iteration and the initial residuals on a logarithmic scale, and draws them again every `[seconds]`. It joins the directories of a continued run. Without a display, it writes `<run-dir>/monitor.png`. It needs `gnuplot`.

8.5 Table of a sweep

```
./post/sweepTable.sh [runs-dir] > sweep.csv    # default □/runs
```

One line for each run. A run that did not start has empty fields.

Columns	Source
`case`, `mesh`, `nCells`	`run.info`
`solver`	`application` in `controlDict`
`Minf`, `pInf`, `Tinf`, `alpha`	`flowConditions`
`status`, `steps`, `tEnd`, `wallSeconds`	Last record of `run.info`
`Cd`, `Cl`, `CmPitch`	Last line of `coefficient.dat`
`window`, `CdMean`, `ClMean`, `CmPitchMean`	Mean over the window of the stop criterion
`resUx` … `resOmega`	Last initial residuals. `resP` is empty for `rhoCentralFoam`.
`yPlusMin`, `yPlusMax`, `yPlusAvgWalls`, `yPlusAvgFins`	y^+ at the last time

With a continued run, the coefficients and residuals come from the last start-time directory only.

8.6 Disk

- The solver log is compressed with `xz` at the end. While the solver runs, the log grows by 1 to 2 kB for each step.
- `residuals` and the coefficients write one line every 50 steps.
- `constant/polyMesh` is a link to the mesh, except with `--refine`.
- `purgeWrite 1` keeps only the last time. `processor*` is removed when `reconstructPar` ends correctly.

CAUTION

Many cases link to one mesh. If you remove or make a mesh again, all the cases that use it lose their mesh.

8.7 Notification

`Allrun` sends a POST request at the end of each run, with the case name, `ok` or `failed`, the time, C_D and the hours of wall time. With ntfy, write the URL of a topic in a file:

```
echo https://ntfy.sh/<topic> > ~/.config/aconcagua/notify
```

The topic is public for each person who knows its name: use a name that is difficult to guess. The environment variable `NOTIFY_URL` has priority over the file. No notification comes if `runCase.sh` is stopped or if the computer stops.

Chapter 9

Comparison with Nektar++

The aim is to run all the Aconcagua cases with OpenFOAM and with Nektar++ and to compare the two solvers. Nektar++ adds p-refinement to the h-refinement of the meshes, so it has more cases.

"The same case" is the same mesh, the same free stream and the same initial field. The schemes, the time integration, the compressible formulation and the turbulence treatment can differ, but they must be as near as possible. This chapter gives what each OpenFOAM run records for the Nektar++ run, and what to measure in the same way in the two solvers.

9.1 What must be the same

Table 9.1: Items that the Nektar++ run must reproduce.

Item	Where it is	Note
Mesh	`meshHash` in `run.info`; `<mesh-dir>/meshHash`	sha256 (16 characters) of `points`, `faces`, `owner`, `neighbour`, `boundary`. With `--refine`, the hash of the refined mesh. The Nektar++ mesh comes from the same `polyMesh`.
Half mesh	`meshInfo`: `sector half`	Symmetry plane $z = 0$. `Aref` is half the frontal area.
Free stream	block `flow` in `run.info`	`Minf`, `alpha`, `pInf`, `Tinf`, `Uinf`, `rhoInf`, `muInf`, the components of **U**.
Reynolds number	`RePerMetre`, `ReD`, `ReL`	From `rhoInf`, `Uinf` and `muInf`, on the diameter and on the body length.
Gas	`Rgas`, `gammaGas`, `Cp`, `RgasThermo`	R is 287.05 in `flowDerived` and 287.10 in the thermophysical model (Section 7.7).
Viscosity	`sutherland (As Ts)`	$\mu = A_s\sqrt{T}/(1 + T_s/T)$.
Conductivity	`PrEucken`	$Pr = 0.690$ from the Eucken correction, not 0.72. Nektar++ must use this value.
Turbulence	`turbulence`, `Prt`, `Ti`, `nuRatio`, `kInlet`, `omegaInlet`	k–ω SST with `nutUSpaldingWallFunction`.

Item	Where it is	Note
Wall	`wallT`, `wallNut`	Adiabatic (`zeroGradient` on T), no slip.
Initial field	`initial`	`uniform free stream`, or the run that was mapped.
References	`FOcommon`	`CofR (0 0 0)` (nose tip); drag and lift in wind axes; `lRef`; `Aref`.

9.2 What to compare

Quantity	OpenFOAM file
History of C_D, C_L, C_m	`forceCoeffs1/*/coefficient.dat`, every 50 steps
Pressure and viscous force	`forces1/*/force.dat`
C_p, τ_w, y^+ along the body	`samples/*/skinZ0_*.csv`, `skinY0_*.csv`
Wake profiles	`samples/*/wake05`, `wake1`, `wake2`, `wake5`
Axis and free-stream check	`samples/*/axis_*.csv`, `freeStream_*.csv`
Skin and symmetry plane	`surfaces/*/` (VTK)
Cost	`cyclesPerCellStep`, `cpuSeconds`, `energyPkgJ`, `GHzEffective` in `run.info`

9.3 How to measure in the same way

- **Degrees of freedom.** OpenFOAM: N_{cells} for each variable. Nektar++: $N_{\text{elements}}\,(P+1)^3$ for each variable on hexahedra (a different count on prisms and tetrahedra). Compare the cost for each degree of freedom and each step, in cycles.
- **Convergence.** Each solver stops with its own criterion and its own step. For the cost to the result, apply one criterion to the two histories of C_D, against the accumulated cost. Do not use the residuals: each code normalizes them differently.
- **Refinement.** Each pair (h, P) of Nektar++ is a case. The cost for each degree of freedom puts them on the same curve as $h1$ to $h3$ of OpenFOAM.
- **Computer.** Use the procedure of Section 8.3, the same computer, the same number of processes and the same binding. Do not divide by the clock frequency: measure the dependence on the clock.

Chapter 10

Flight data

The TeleMega recorded the IREC flight in Midland. `flightData/cdFlight.py` calculates from these data the atmosphere of that day, the speed, the drag coefficient in the coast phase and the conditions of the sweep. It plots the result against the CFD runs and OpenRocket.

10.1 Command

```
python3 flightData/cdFlight.py [--csv AconcaguaIREC_TeleMega.csv] [--mass 24.52]
        [--runs □/runs] [--ork rocket.ork] [--alphas 0,8] [--out cdFlight.png]
```

Option	Default	Function
`--csv`	`flightData/AconcaguaIREC_TeleMega.csv`	TeleMega data.
`--mass`	24.52 kg	Mass after burnout, from the OpenRocket simulation.
`--runs`	`~/runs`	Runs to plot. The script uses the last segment of each run.
`--ork`	`~/Downloads/207 - Aconcagua.ork`	OpenRocket file for its C_D curve. Optional.
`--alphas`	`0,8`	Angles of the runs to plot.
`--tanchor`	18 s	Time of the speed anchor.
`--out`	`flightData/cdFlight.png`	Plot.

10.2 Method

Figure 10.1 shows the method. All values come from the telemetry, except the mass.

1. **Atmosphere.** The pad pressure p_0 is the mean before launch. During the descent, the GPS gives the height above the pad, $h = \sqrt{\texttt{pad_range}^2 - \texttt{pad_dist}^2}$, and the barometer gives the pressure without shocks. The script fits T_0 and the lapse rate L in

$$p = p_0 \left(1 - \frac{L\,h}{T_0}\right)^{g/(RL)}.$$

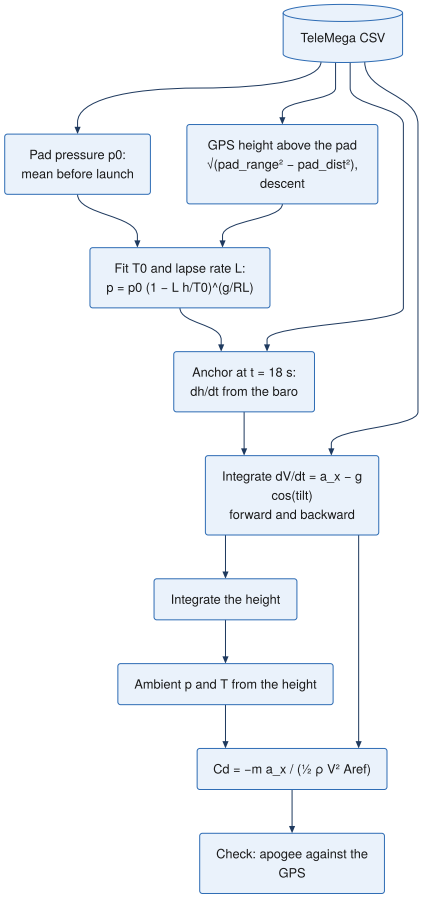


Figure 10.1: Calculation of the flight drag coefficient.

Result: $p_0 = 90\,565\,Pa$, $T_0 = 305.0\,K$, $L = 7.2\,K/km$, from 215 GPS points, with an rms error of 0.3 % in $\ln p$. The air was approximately 20 K warmer than the ISA at all heights.

2. **Speed.** The script does not use the column `speed` of the TeleMega: its filter converts pressure to height with the ISA, and it gives approximately 10 % less in the coast. Above $M \approx 0.8$ the barometer is not used either, because the static ports are behind shocks. The script integrates the axial accelerometer, forward and backward from $t = 18\,s$:

$$\frac{dV}{dt} = a_x - g\cos\theta_{\text{tilt}}.$$

At $t = 18\,s$ ($M \approx 0.65$) the speed comes from the derivative of the barometric height with the fitted atmosphere.

3. **Height and ambient state.** The script integrates $V\cos\theta$ for the height, and calculates p and T from the height with the fitted atmosphere.

4. **Drag.** In the coast, a_x is the axial specific force, so $D = -m\,a_x$ and

$$C_D = \frac{-m\,a_x}{\frac{1}{2}\rho V^2 A_{\text{ref}}}, \qquad A_{\text{ref}} = \pi\,(0.0755\,m)^2.$$

The script uses the data from $t = 4.0\,s$, when the residual thrust ends.

5. **Check.** The integrated apogee is 8137 m above the pad. The GPS gives 8083 m (0.7 % difference).

10.3 Results

Table 10.1: C_D of the flight and of the CFD at $\alpha = 0$ (first sweep, ISA conditions, 2026-10-17).

M	Flight	CFD	CFD/flight	OpenRocket
0.3	0.369	0.376 ($h0$)	1.02	0.336
0.6	0.337	0.356 ($h0$)	1.05	0.337
0.8	0.320	0.349 ($h0$), 0.339 ($h1$)	1.09, 1.06	0.378
≈1.03	≈0.70 (peak)	–	–	≈0.44
1.2	≈0.64	–	–	≈0.43
1.8	0.388	0.438 (pimple start only)	1.13	0.389

- From $M = 0.3$ to 0.8 the CFD is 2 to 9 % above the flight, and it decreases from $h0$ to $h1$.
- In the flight, C_D doubles from $M = 0.9$ to 1.0. OpenRocket does not show this peak. No CFD run is in this range yet.
- The flight value at $M = 1.8$ is 13 % below the pimple start run. That run is not a result of the central template.

10.4 Angle of attack in the flight

The rocket rolls at approximately 4 revolutions per second in the coast. The mean lateral acceleration follows the square of the roll rate with a constant radius of approximately 2.8 cm:

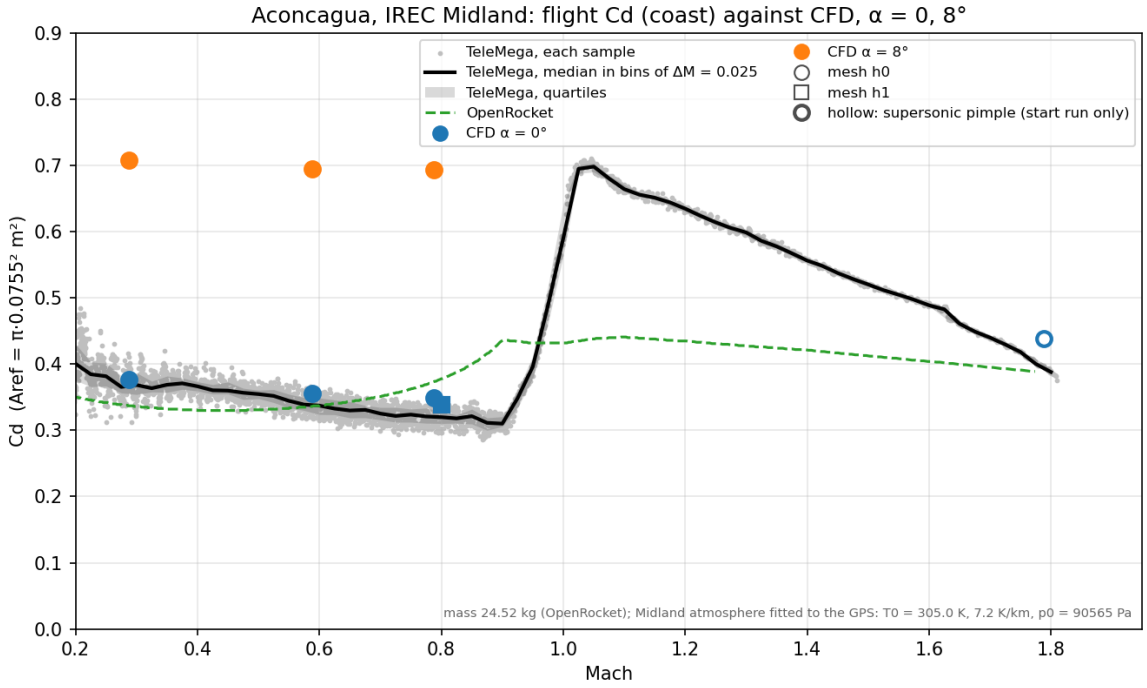


Figure 10.2: C_D of the flight (coast) against the runs of the first sweep (ISA conditions, `~/runs.sin-medicion`) and OpenRocket.

it is the centripetal acceleration of a sensor off the axis. An aerodynamic normal force has a fixed direction in space, so in the body axes it turns with the roll. The oscillation of the lateral acceleration at the roll frequency gives $C_N \approx 0.01$ to 0.03. With $C_{N\alpha} \approx 18\,/rad$ from the CFD, $\alpha < 0.2°$ in all the coast. The sensor noise is in this value, so it is an upper limit.

10.5 Uncertainty

Source	Effect
Mass	C_D is proportional to the mass, which is a design value of OpenRocket, not a measurement.
Wind	No data. The speed is relative to the ground.
Flight path	The gravity term uses the attitude as the direction of the flight path. With $\alpha < 0.2°$ the error is small.
Atmosphere	The fit gives one lapse rate from the pad to 8 km. The local layers differ from the fit by up to 7 K.

Chapter 11

Validation

This chapter gives the procedures that compare the results with references. Each reference has its formula and its constants, so that you can calculate it by hand. Do the procedures in this sequence:

1. The checks of Section 11.1, on each run.
2. Subsonic flow (Section 11.2). There are analytical references.
3. Transonic and supersonic flow (Section 11.3).
4. The flight (Chapter 10).
5. Mesh convergence (Section 11.4), before you publish a value.

11.1 Checks on each run

Table 11.1: Checks on each run.

Check	Correct result	Cause if not correct
C_L, `CmPitch` at $\alpha = 0$	≈ 0	The mesh is not symmetric, or the run did not converge.
`CmRoll` at $\alpha = 0$	≈ 0	The fin meshes are not equal.
y^+ on each wall patch	Near Table 4.7, and the same on all meshes of a study	The first cell changed (Section 4.3.4).
Minimum and maximum of T	Possible: at most near the stagnation temperature, not near 0	The start transient is not resolved.
Iterations	More than `minIter`, and the stop criterion met	The run stopped before the flow was steady.
C_N from $\alpha = 2°$ to $8°$	Linear in α ($C_{N\alpha} \approx 18\ /rad$ on $h0$)	The run did not converge, or y^+ on the fins is out of range.
Machine record	`suspendedSeconds` 0, `profiles performance`	The cost of the run is not usable (Section 8.3).

11.2 Subsonic flow: Barrowman

The references are the equations of Barrowman [15], Barrowman and Barrowman [16], which OpenRocket uses. Fleeman [17, ch. 2] gives equivalent equations for the fins and the fin-body interference. The coefficients refer to the body cross-section $A = \pi d^2/4$ and are for each radian.

11.2.1 Geometry data

Symbol	Value	Source
d	0.151 m	`lRef`
L	2.955 m	`lBody`
r	0.0755 m	`rBody`
s, exposed semi-span	$0.2355 - 0.0755 = 0.160\,m$	`finTipR` − `rBody`
c_r, c_t, l_m	–	Measure on the fin drawing or in `makeRocket.py`
$(d_{\text{base}}/d)^2$	0.53	$d_{\text{base}} = 0.110\,m$

NOTE

`finX1` − `finX0` $= 0.401\,m$ is the x extent of the fin, not the root chord.

11.2.2 Normal force and centre of pressure

Component	$C_{N\alpha}$ [1/rad]	x_{cp} from the nose tip
Nose	2.0 for all noses of revolution	$0.466\,L_{\text{nose}}$ for an ogive
Cylinder	≈ 0 (slender-body theory)	–
Boattail	$2[(d_{\text{base}}/d)^2 - 1] = -0.94$	From the drawing
Fins, $N = 4$	Equation below	From the drawing

$$C_{N,\text{fins}} = K_{fb}\,\frac{4N(s/d)^2}{1 + \sqrt{1 + \left(\dfrac{2l_m}{c_r + c_t}\right)^2}}, \qquad K_{fb} = 1 + \frac{r}{s + r}.$$

11.2.3 Procedure

1. Run an angle-of-attack sweep from $0°$ to $6°$.
2. Calculate the slope of C_N against α in radians. Use the force normal to the body axis, not C_L: C_L is normal to the free stream, $C_N = C_L \cos\alpha + C_D \sin\alpha$.
3. Compare the slope with the sum of the components.
4. For the breakdown, add one `forceCoeffs` function object for each patch and compare each term with its reference.

11.2.4 Drag at zero angle of attack

Compare the components of the drag, not only the total.

Term	Reference
Skin friction	$C_f\, S_{\text{wet}}/A_{\text{ref}}$, with $C_f = 0.0576\, Re_L^{-0.2}$ (turbulent flat plate, 18, ch. 21)
Body form factor	$1 + 1/(2\, L/d)$, with $L/d = 19.57$
Fin form factor	$1 + 2\, t/c$
Base drag	Hoerner [19, ch. 3], with $(d_{\text{base}}/d)^2$

Re_L for each case is in `run.info` (`ReL`). The C_f correlation is for incompressible flow. `forces1` gives the pressure and the viscous parts in different columns: compare the viscous part with the skin friction term.

11.3 Transonic and supersonic flow

No analytical reference applies to the full rocket. Use these methods, from the smallest to the largest work.

1. **Tutorial of the template.** Run `$FOAM_TUTORIALS/compressible/rhoCentralFoam/biconic25-55Run35` and compare it with its published data. This validates the solver and the flux scheme, and separates a configuration error from a mesh error.
2. **Oblique shock on a wedge.** The θ–β–M relation is exact [20, ch. 4]:

$$\tan\theta = 2\cot\beta\,\frac{M^2\sin^2\beta - 1}{M^2(\gamma + \cos 2\beta) + 2}.$$

   The tutorial `rhoCentralFoam/wedge15Ma5` is this case. Compare the shock angle and the pressure ratio.
3. **Flight.** Chapter 10. The flight C_D rises from 0.31 at $M = 0.9$ to approximately 0.70 at $M = 1.03$, and decreases to 0.39 at $M = 1.8$. The CFD must show this rise. If it does not, the mesh is too coarse where the shock occurs.
4. **OpenRocket.** Semi-empirical. It finds an error of a factor of 2. It does not support a difference of 5 %, and it does not show the transonic peak of the flight.

11.4 Mesh convergence

11.4.1 Procedure

1. Make three meshes with only `maxCellSize` changed (Section 6.6.2).

2. Run the same flight condition on the three meshes.

3. Measure y^+ on each mesh. It must stay in the same range.

4. Calculate the representative cell size $h = N^{-1/3}$ of each mesh, and the ratio of each pair, $r = h_{\text{coarse}}/h_{\text{fine}}$.

5. Calculate the observed order p, the extrapolated value and the grid convergence index with the method of Celik et al. [4].

11.4.2 Equations

With f_1, f_2, f_3 the results on the fine, the medium and the coarse mesh, and a constant ratio r:

$$p = \frac{\ln\left[(f_3 - f_2)/(f_2 - f_1)\right]}{\ln r}, \qquad f_{\text{ext}} = f_1 + \frac{f_1 - f_2}{r^p - 1}, \qquad \text{GCI}_{12} = 1.25\,\frac{|(f_1 - f_2)/f_1|}{r^p - 1}.$$

The ratios of the sweep meshes are not equal (1.25 and 1.33, Section 6.6.2). Use the iterative form of Celik et al. [4] for unequal ratios. If $(f_3 - f_2)/(f_2 - f_1)$ is negative, the convergence is oscillatory and the equations do not apply [21].

11.5 Record

For each run that you publish, keep these files with the results:

File	Function
`<mesh-dir>/system/meshDict`, `meshInfo`, `log.checkMesh`	Make the mesh again; its record and quality.
The STL file, or its commit	Geometry.
`<run-dir>/run.info`	Case, commit, mesh fingerprint, flight condition, computer and cost.
`<run-dir>/system/flowConditions`, `config.json`	Flight condition, ramp and stop criterion.
`postProcessing/`	Coefficients, y^+, samples.

Chapter 12

Troubleshooting

12.1 Symptoms and remedies

Table 12.1: Symptoms, causes and remedies.

Symptom	Cause	Remedy
A script stops with `source the OpenFOAM bas hrc first`.	The OpenFOAM environment is not loaded.	Chapter 2.
`Allmesh` shows negative cell volumes.	The geometry or `meshDict` does not agree with Table 3.4.	Do not use the mesh. Change the geometry or the levels and make a new mesh.
`newCase.sh` stops with `needs jinja2`.	`jinja2-cli` is not installed.	`pipx install jinja2-cli`.
The domain of a new mesh is incorrect, or a number has a comma.	A script calculates with `awk` in a locale with a decimal comma.	Section 12.5.
`rhoCentralFoam` stops with a floating point exception in the first 300 iterations.	Start from the uniform free stream: the base empties.	Start from a pimple run (Section 7.15).
The Courant number in the log is of the order of 10^7.	Normal with `localEuler`.	Section 12.4.
A run started with `--init` has the C_D of a start from rest.	`mapFields` failed.	Section 12.3.
C_D of the central template increases without limit.	Open defect.	Section 12.2.
`cpuMigrations` is large, or `taskset` shows all CPUs for a process.	The binding does not work.	Section 5.2.3. Use the current `Allrun`.
`suspendedSeconds` is not 0, or the wall time is much larger than the CPU time.	The computer suspended, for example when the lid closed.	Section 2.3, step 3. The coefficients are correct; the cost is not.

Symptom	Cause	Remedy
`GHzEffective` is near 2 on `nemepad`, or `profiles` is not `performance`.	The CPU profile changed.	Section 2.3, step 1.
No `cycles` in `run.info`; `Allrun` shows `perf cannot count cycles`.	`perf_event_paranoid` is 2 or more, for example after a restart.	Section 2.3, step 2.
The computer stops under load.	A defective memory module (`newton`, DIMM_A1).	Do not use the computer until the module is replaced.
The run stops with no memory, or the computer swaps.	The mesh is too large for the memory (approximately 3 GB for each million cells).	Use another computer for $h3$.
`pkill -f` stops your own shell.	The command line of the shell contains the pattern.	Stop the process by its number with `kill`.
A sweep reads incorrect lines after an edit of its file.	The sweep file was edited while `runCase.sh` ran.	Stop `runCase.sh`, edit, start it again.
A shell script does unexpected things after an edit.	`sh` reads a script while it runs it.	Do not edit a script that runs.
An OpenFOAM dictionary gives a parse error near its banner.	The banner comment contains `*/`, which ends the comment early.	Remove `*/` from the comment.
A script continues after an application failed.	`runApplication` returns 0 also when the application stops with `FOAM FATAL ERROR`.	Read the log of each application, as `Allmesh` and `Allrun` do.
OpenFOAM is very slow on WSL.	The case is in `/mnt/c`.	Move the mesh and run directories to the Linux file system.

12.2 Central template: drag increases without limit

Status: open (2026-10-18). Do not run the supersonic sweep.

Run `m18-a00-h0`: `rhoCentralFoam` on $h0$ at $M = 1.8$, started from the pimple run at the same condition (ISA conditions of the first sweep).

The run stopped at iteration 584 because the computer stopped, not the solver. The wall shear stress increases by a factor of 3. The flight gives $C_D = 0.39$ at this Mach number. Items to examine:

- The wall conditions and the wall functions of the two templates (`nut`, `k`, `omega`, `T`, `alphat`).
- The transfer of the fields with `mapFields` and `changeDictionary`, including the turbulence fields.

Table 12.2: Pimple start against the central run, $h0$, $M = 1.8$.

	Pimple at the end	Central, iteration 550
C_D	0.437	0.894, still increasing
Viscous force, x	266 N	828 N
Pressure force, x	445 N	604 N
Mean y^+ on `walls`	16.6	33.7

- The flux scheme and the limiter at the wall.
- The difference between the start transient and the steady state.

12.3 mapFields on the same mesh

Symptom. `mapFields` stops with a segmentation fault at the field `Ma`, when the source and the target are the same mesh ($h0 \to h0$, 2026-10-17). Between different meshes ($h0 \to h1$) it works.

Effect before the correction. `Allrun` looked only for `FOAM FATAL`. The run started from the uniform free stream, and the log said that the mapping was correct.

Current behaviour. `Allrun` stops if the log of `mapFields` or `changeDictionary` does not end with `End`.

Open item. The cause of the fault is not known. The supersonic chain needs this mapping (pimple $\to$ central on $h0$). On the same mesh, a copy of the fields of the source run is an alternative that does not interpolate.

12.4 The Courant number in the log

With `localEuler`, `rhoCentralFoam` calculates the Courant number with `deltaT 1` of `controlDict`, so it writes values of approximately 10^7. `rhoPimpleFoam` does not write it. Read `Flow time scale min/max`. This incorrect reading kept the central template on a global `Euler` step until 2026-10-12.

12.5 Decimal comma in scripts

With the locale `es_ES` (`newton`), `awk` reads and writes numbers with a decimal comma. `Allmesh` once gave `11,775` to `surfaceGenerateBoundingBox` and made an incorrect domain. Each script that calculates with `awk` sets `LC_ALL=C`: `Allmesh`, `Allrun` and the calculation of the block `flow` in `newCase.sh`. Do the same in a new script.

12.6 Rules for a change to the scripts

- Measure before you propose a change. A change without a number is not complete.
- Do not put a Python script in a bash here-document: a `\n` in an f-string becomes a real new line and breaks the file.

- After a change to a script, run one short case and examine `run.info`.
- Change this guide in the same commit.

Appendix A

Quick reference

A.1 Commands

```
source /usr/lib/openfoam/openfoam2412/etc/bashrc

# geometry and one mesh
python3 mesh/makeRocket.py mesh/stl/Aconcagua_biconvex117_roll45.stl --finRoll 45
mesh/Allmesh □/meshes/h1 --maxCellSize 1.6 --nLayers 10 --thicknessRatio 1.2
./post/firstLayer.py □/meshes/h1

# one case
./newCase.sh □/runs/test --mesh □/meshes/h1 --solver pimple --Minf 0.8 --np 8
cd □/runs/test && ./Allrun 8

# sweep
./runMesh.sh meshes.txt □/meshes
nohup ./runCase.sh sweepSubsonic.nemepad.txt > □/log.sweepSubsonic.nemepad 2>&1 &

# results
./post/monitor.sh □/runs/test
./post/sweepTable.sh □/runs > sweep.csv
python3 flightData/cdFlight.py

# check the binding and the measurement of a running case
for p in $(pgrep -x rhoPimpleFoam); do taskset -cp $p; done
tail -3 □/runs/<case>/log.machine
```

A.2 Run names

`m<M>-a<alpha>-h<n>[-pimple]`: `m08` is $M = 0.8$, `a04` is $\alpha = 4°$, `h1` is the mesh. The suffix `-pimple` marks the `rhoPimpleFoam` start run of a supersonic chain.

A.3 Key values

Item	Value
Reference area, full rocket	$A_{\text{ref}} = \pi\,(0.0755)^2 = 0.0179081\,m^2$
Reference length	$l_{\text{ref}} = d = 0.151\,m$
Moment reference	Nose tip, $(0, 0, 0)$
Rocket length	2.955 m
First cell on the body	$t_1 \approx 0.10\,mm$, $y \approx 0.05\,mm$
Stop criterion	$\lvert\overline{C_D} - C_D\rvert < 0.007$ over 1000 iterations
Memory	approximately 3 GB for each million cells
Midland atmosphere	$p_0 = 90\,565\,Pa$ at 896 m MSL, $T_0 = 305.0\,K$, 7.2 K/km

Appendix B

Glossary

B.1 Terms and abbreviations

Term	Definition
Binding	The assignment of each MPI process to fixed CPU cores.
Boattail	The cone at the rear of the body, from the body diameter to the base.
Chain	A sequence of runs at one flight condition, each started from the run on the coarser mesh.
Coast	The flight phase after the motor burnout, $t > 4\,s$.
CFD	Computational fluid dynamics.
DOF	Degree of freedom: one unknown of the discrete system.
GCI	Grid convergence index [4].
IPC	Instructions per cycle.
ISA	International Standard Atmosphere.
LTS	Local time stepping (`localEuler`).
Mesh $h0$ to $h3$	The meshes of the sweep (Table 4.6).
MSL	Mean sea level.
RANS	Reynolds-averaged Navier–Stokes equations.
Run	One case directory and its executions.
SST	Shear-stress transport (k–ω SST model).
Sweep	A file with one run on each line, and its execution by `runCase.sh`.
Template	`case-central/` or `case-pimple/`: the files from which `newCase.sh` makes a case.
TVD	Total variation diminishing.
y^+	Distance of the first cell centre from the wall, in wall units: $y\,u_\tau/\nu$.

B.2 Symbols

Symbol	Definition	Unit
A_{ref}	Reference area	m^2
a	Speed of sound	m/s
C_D, C_L, C_N	Drag, lift and normal-force coefficients	–
C_m	Pitching-moment coefficient about the nose tip	–
C_p	Pressure coefficient	–
c_p, c_v	Specific heats	J/(kg K)
d	Body diameter	m
k, ω	Turbulent kinetic energy, specific dissipation rate	m^2/s^2, $1/\text{s}$
M	Mach number	–
p, T, ρ	Pressure, temperature, density	Pa, K, kg/m^3
Pr, Pr_t	Laminar and turbulent Prandtl numbers	–
R	Gas constant of air	J/(kg K)
Re	Reynolds number	–
t_1	Thickness of the first wall cell	m
α, β	Angle of attack, angle of sideslip	deg
γ	Ratio of specific heats	–
μ, ν, ν_t	Dynamic, kinematic and eddy viscosity	Pa s, m^2/s
τ_w	Wall shear stress	Pa

Bibliography

[1] ISO/IEC/IEEE. ISO/IEC/IEEE 26514:2022 systems and software engineering – design and development of information for users, 2022.

[2] ASD Simplified Technical English Maintenance Group. ASD-STE100 simplified technical english: International specification for the preparation of technical documentation in a controlled language, 2021. Issue 8.

[3] J. Martí Valverde and C. Stemmer. OpenFOAM ToolChain for the aerodynamics of sounding rockets. Technical report, Zenodo, 2026.

[4] I. B. Celik, U. Ghia, P. J. Roache, C. J. Freitas, H. Coleman, and P. E. Raad. Procedure for estimation and reporting of uncertainty due to discretization in CFD applications. *Journal of Fluids Engineering*, 130:078001, 2008.

[5] A. Kurganov and E. Tadmor. New high-resolution central schemes for nonlinear conservation laws and convection-diffusion equations. *Journal of Computational Physics*, 160: 241–282, 2000.

[6] A. Kurganov, S. Noelle, and G. Petrova. Semidiscrete central-upwind schemes for hyperbolic conservation laws and Hamilton–Jacobi equations. *SIAM Journal on Scientific Computing*, 23(3):707–740, 2001.

[7] C. J. Greenshields, H. G. Weller, L. Gasparini, and J. M. Reese. Implicit and explicit schemes for flows of aerodynamic and turbomachinery fluids. *International Journal for Numerical Methods in Fluids*, 63:1–21, 2010.

[8] B. van Leer. Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov's method. *Journal of Computational Physics*, 32:101–136, 1979.

[9] F. R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.

[10] F. R. Menter, M. Kuntz, and R. Langtry. Ten years of industrial experience with the SST turbulence model. In *Turbulence, Heat and Mass Transfer 4*, pages 625–632, 2003.

[11] D. B. Spalding. A single formula for the law of the wall. *Journal of Applied Mechanics*, 28 (3):455–458, 1961.

[12] W. M. Kays. Turbulent Prandtl number – where are we? *Journal of Heat Transfer*, 116: 284–295, 1994.

[13] F. M. White. *Viscous Fluid Flow*. McGraw-Hill, 3rd edition, 2006.

[14] T. J. Poinsot and S. K. Lele. Boundary conditions for direct simulations of compressible viscous flows. *Journal of Computational Physics*, 101:104–129, 1992.

[15] J. S. Barrowman. The practical calculation of the aerodynamic characteristics of slender finned vehicles. Master's thesis, Catholic University of America, 1967.

[16] J. S. Barrowman and J. A. Barrowman. A method for calculating the static margin of a slender missile, 1966.

[17] E. L. Fleeman. *Tactical Missile Design*. AIAA, 2nd edition, 2006.

[18] H. Schlichting. *Boundary-Layer Theory*. McGraw-Hill, 7th edition, 1979.

[19] S. F. Hoerner. *Fluid-Dynamic Drag*. Hoerner Fluid Dynamics, 1965.

[20] J. D. Anderson. *Modern Compressible Flow*. McGraw-Hill, 3rd edition, 2003.

[21] P. J. Roache. Perspective: a method for uniform reporting of grid refinement studies. *Journal of Fluids Engineering*, 116:405–413, 1994.